InternMatch — Complete Project Specification & Build Prompt

> ***What this document is:*** *A fully articulated, production-ready specification for building InternMatch from scratch. Every feature, every page, every API endpoint, every database table, every algorithm, and every design decision is defined here. A developer (or AI) should be able to read this document and build the entire system without asking a single clarifying question.*

PROJECT OVERVIEW

InternMatch is an AI-powered internship matching web platform that connects university students with internship opportunities using intelligent skill-based matching. The platform serves three user roles — **Students**, **Employers**, and **Administrators** — each with distinct dashboards, capabilities, and workflows.

The core differentiator is an **AI matching engine** that automatically extracts skills from uploaded resumes, compares them against internship requirements, and generates percentage-based match scores to recommend the best-fit internships to students and the best-fit candidates to employers.

TECH STACK

Layer	Technology	Rationale
Frontend	React.js 18+	Component-based SPA, large ecosystem
Styling	Tailwind CSS 3+	Utility-first, rapid UI development, built-in dark mode support
State Management	React Context API + useReducer	Sufficient for auth state and theme; no Redux overhead needed
Routing	React Router v6	Standard for React SPAs
HTTP Client	Axios	Interceptors for JWT, clean API layer
Icons	Lucide React	Clean, consistent icon set
Charts	Recharts	React-native charting for dashboards/analytics
Notifications (UI)	React Toastify	Toast notifications for user feedback
Backend	Node.js + Express.js	JavaScript everywhere, simple REST API framework
Database	MySQL 8.0	Relational, matches the ER diagram from Part 1, free
ORM/Query	mysql2 (raw queries with prepared statements)	Full SQL control, no ORM abstraction overhead

Layer	Technology	Rationale
Authentication	JWT (jsonwebtoken) + bcryptjs	Stateless auth, industry standard
File Uploads	Multer	Standard Express middleware for multipart/form-data
Resume Parsing	pdf-parse (PDF) + mammoth (DOCX)	Extract raw text from uploaded resumes
AI/NLP — LLM	Google Gemini API (gemini-2.5-flash-lite, free tier) via `@google/generative-ai`	LLM-powered skill extraction from resumes — real NLP, not keyword matching. Free tier: 15 RPM, 1000 RPD — more than sufficient for a university project
AI/NLP — Embeddings	Google Gemini Embedding API (`text-embedding-004`) via `@google/generative-ai`	Generates 768-dimensional semantic vector embeddings for internship descriptions and search queries. Powers NLP semantic search and semantic matching. Free tier: 1500 RPM, 14400 RPD
AI/NLP — Similarity	cosine-similarity (npm) or manual cosine calculation	Computes cosine similarity between embedding vectors for semantic search ranking and profile-to-internship semantic matching
Email	Nodemailer + Gmail SMTP (free) or Mailtrap (free tier for dev)	Transactional emails for notifications and password recovery
Real-time Messaging	Socket.IO	WebSocket-based real-time chat between students and employers
Validation	express-validator (backend) + simple controlled components with inline validation (frontend)	Backend does the real validation; frontend shows user-friendly errors. No need for a form library in a project this size
Environment Config	dotenv	Secure configuration management
Dev Tooling	nodemon, concurrently	Auto-restart server, run frontend+backend simultaneously
Version Control	Git + GitHub	Collaboration and source code management

---

DATABASE SCHEMA

All table and column names use snake_case. Most tables include `created_at` and/or `updated_at` where appropriate, but some tables use domain-specific timestamp fields (e.g., `applied_date`, `status_updated_at`, `viewed_at`, `expires_at`) when semantically clearer. Table `user` renamed to `users` to avoid reserved word issues. All corrections from database review applied.

Enforcement model: The schema uses DB-level constraints (ENUM, CHECK, composite FKs, UNIQUE) for structural integrity rules that must never be violated. Some classification fields — `employer.industry`, `notification.type`, `notification.reference_type`, `system_log.action`, `report.report_type`, and `admin.access_level` — are stored as VARCHAR and enforced by backend constants only. This is an intentional trade-off: VARCHAR allows extending allowed values without a schema migration, but it means the DB alone does not guarantee valid values for those fields. Backend validation is mandatory for all of them.

Table: `users`

Column	Type	Constraints	Notes
user_id	INT	PK, AUTO_INCREMENT	

Column	Type	Constraints	Notes
full_name	VARCHAR(100)	NOT NULL	
email	VARCHAR(150)	NOT NULL, UNIQUE	Used for login
password	VARCHAR(255)	NOT NULL	bcrypt hashed, salt rounds = 12
role	ENUM('student','employer','admin')	NOT NULL	Determines which sub-table to join
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE	**Suspension flag** (not soft-delete). Admin can deactivate a user to temporarily block login without deleting their data. Deactivated users cannot log in but their records remain in the system. For permanent removal, use the DELETE endpoint which hard-deletes the user row and cascades.
profile_picture	VARCHAR(255)	NULLABLE	Path to uploaded image
created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	
updated_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	
token_version	INT	NOT NULL, DEFAULT 0	Incremented on password change or forced logout. The JWT payload includes `tokenVersion` at sign time. Auth middleware compares JWT's `tokenVersion` against the DB value — if they don't match, the token is rejected (forces re-login). This ensures that changing your password invalidates all existing sessions.

Indexes: `UNIQUE(email)`, `INDEX(role, is_active)`

Table: `student`

Column	Type	Constraints	Notes
user_id	INT	PK, FK → users.user_id ON DELETE CASCADE	
major	VARCHAR(100)	NOT NULL	
university	VARCHAR(150)	NOT NULL	
university_start_date	DATE	NULLABLE	When the student started university
gpa	DECIMAL(3,2)	NULLABLE, CHECK (gpa BETWEEN 0.00 AND 4.00)	Scale of 4.00
graduation_year	INT	NOT NULL, CHECK (graduation_year BETWEEN 2000 AND 2100)	

Column	Type	Constraints	Notes
graduation_status	—	NOT stored in DB — computed on read	Derived at query time: if `graduation_year < YEAR(CURDATE())` then 'graduated', else 'enrolled'. Computed in the backend service layer or as a SQL expression: `IF(graduation_year < YEAR(CURDATE()), 'graduated', 'enrolled') AS graduation_status`. Never stored because it would go stale on year rollover.
bio	TEXT	NULLABLE	Short personal statement
linkedin_url	VARCHAR(255)	NULLABLE	
github_url	VARCHAR(255)	NULLABLE	
instagram_url	VARCHAR(255)	NULLABLE	
phone	VARCHAR(20)	NULLABLE	
primary_resume_id	INT	NULLABLE	Points to the student's single resume (NULL if no resume uploaded)
updated_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	

Ownership constraint on primary_resume_id: A simple FK `(primary_resume_id) → resume(resume_id)` does not guarantee the resume belongs to this student. Fix: use a **composite foreign key** that enforces ownership at the DB level:

```
-- On resume table, add a unique index to enable the composite FK:
ALTER TABLE resume ADD UNIQUE INDEX idx_resume_ownership (resume_id, student_user_id);

-- On student table, add the composite FK with ON DELETE RESTRICT (not SET NULL):
ALTER TABLE student ADD CONSTRAINT fk_student_primary_resume
  FOREIGN KEY (primary_resume_id, user_id)
  REFERENCES resume (resume_id, student_user_id)
  ON DELETE RESTRICT;
```

This guarantees that a student can only set a resume as primary if `resume.student_user_id` matches `student.user_id`. The DB rejects any attempt to set another student's resume as primary.

Why ON DELETE RESTRICT, not ON DELETE SET NULL: MySQL's `ON DELETE SET NULL` on a composite FK tries to null ALL referencing columns — including `student.user_id` which is the PK and cannot be null. That would crash. Instead, use RESTRICT and handle in app code with a **database transaction for DB state** and **reference-counted file cleanup** for storage:

```
-- Resume replacement flow (student uploads new resume, replacing existing one):
START TRANSACTION;
-- Step 1: Clear primary_resume_id
UPDATE student SET primary_resume_id = NULL WHERE user_id = ?;
-- Step 2: Clear resume_id on any applications referencing the old resume
-- (required because application.resume_id also uses composite FK with ON DELETE RESTRICT)
-- The submitted_resume_path and submitted_resume_filename snapshots are preserved, so no data is
lost.
UPDATE application SET resume_id = NULL WHERE resume_id = ? AND student_user_id = ?;
-- Step 3: Delete the old resume record
DELETE FROM resume WHERE resume_id = ? AND student_user_id = ?;
-- Step 4: Insert the new resume record
INSERT INTO resume (student_user_id, file_path, original_filename, file_type, extracted_text)
VALUES (?, ?, ?, ?, ?);
-- Step 5: Set the new resume as primary
UPDATE student SET primary_resume_id = LAST_INSERT_ID() WHERE user_id = ?;
COMMIT;
-- Step 6 (AFTER commit, outside transaction): File cleanup with reference counting.
-- Check if any application still references the old file path via submitted_resume_path:
--   SELECT COUNT(*) FROM application WHERE submitted_resume_path = ?
-- If count > 0 → do NOT delete the old file. It is still needed for submitted application
downloads.
-- If count = 0 → safe to delete the old physical file.
-- If DB transaction fails → ROLLBACK, do not touch any files, keep old resume intact.
-- If DB succeeds but file deletion fails → log the orphaned file path for scheduled cleanup.
```

Resume file retention rule: A physical resume file on disk MUST NOT be deleted as long as any `application.submitted_resume_path` references it. This ensures that submitted application resumes remain downloadable even after the student replaces or deletes their current resume. File cleanup is always reference-counted: check for remaining references before deleting.

Table: `employer`

Column	Type	Constraints	Notes
user_id	INT	PK, FK → users.user_id ON DELETE CASCADE	
company_name	VARCHAR(150)	NOT NULL	
industry	VARCHAR(100)	NOT NULL	**Allowed values (enforced by backend validation):** `Technology`, `Finance`, `Healthcare`, `Education`, `Marketing`, `Engineering`, `Other`. The frontend shows these as a dropdown. The backend rejects any value not in this list. Stored as VARCHAR for flexibility to extend the list later without a schema migration — but the current allowed list is a backend constant.
company_size	ENUM('1-50','51-200','201-500','500+')	NOT NULL	
company_description	TEXT	NULLABLE	About the company

Column	Type	Constraints	Notes
company_logo	VARCHAR(255)	NULLABLE	Path to uploaded logo
website_url	VARCHAR(255)	NULLABLE	Company website
linkedin_url	VARCHAR(255)	NULLABLE	Company LinkedIn page
twitter_url	VARCHAR(255)	NULLABLE	Company Twitter/X page
facebook_url	VARCHAR(255)	NULLABLE	Company Facebook page
instagram_url	VARCHAR(255)	NULLABLE	Company Instagram page
location	VARCHAR(150)	NULLABLE	Company headquarters
updated_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	

Table: `admin`

Column	Type	Constraints	Notes
user_id	INT	PK, FK → users.user_id ON DELETE CASCADE	
access_level	VARCHAR(50)	NOT NULL, DEFAULT 'SuperAdmin'	

Table: `internship`

Column	Type	Constraints	Notes
internship_id	INT	PK, AUTO_INCREMENT	
employer_user_id	INT	NOT NULL, FK → employer.user_id ON DELETE CASCADE	If employer is deleted, their internships are deleted too
title	VARCHAR(200)	NOT NULL	
description	TEXT	NOT NULL	Full description, responsibilities, etc.
location	VARCHAR(150)	NOT NULL	
duration_months	INT	NOT NULL, CHECK (duration_months > 0 AND duration_months <= 24)	
work_type	ENUM('remote','hybrid','on-site')	NOT NULL	Single source of truth for work arrangement
salary_min	DECIMAL(10,2)	NULLABLE, CHECK (salary_min >= 0)	Optional salary range

Column	Type	Constraints	Notes
salary_max	DECIMAL(10,2)	NULLABLE, CHECK (salary_max >= 0 AND (salary_min IS NULL OR salary_max >= salary_min))	
status	ENUM('pending_approval','active','closed','rejected')	NOT NULL, DEFAULT 'pending_approval'	Admin must approve
admin_review_note	TEXT	NULLABLE	Reason for rejection if any
deadline	DATE	NULLABLE	Application deadline
created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	
updated_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	

Indexes: `INDEX(employer_user_id)`, `UNIQUE(internship_id, employer_user_id)` (supports composite FK from internship_invitation), `INDEX(status)`, `INDEX(status, deadline)`, `INDEX(created_at)`

Deadline enforcement (auto-close): Internships past their deadline are automatically closed. Implemented as a **dual approach**:

1. **On-access auto-close (primary):** The `GET /internships` and `GET /internships/:id` endpoints execute a quick UPDATE before returning results: `UPDATE internship SET status = 'closed' WHERE status = 'active' AND deadline IS NOT NULL AND deadline < CURDATE()`. This is lightweight (indexed on `status, deadline`) and ensures data stays consistent.
2. **Scheduled cleanup (secondary, optional):** A daily cron job (or `setInterval` on server startup running every hour) closes all past-deadline active internships in bulk using the same UPDATE query. This catches any that the on-access check missed.
3. **Application rejection:** `POST /applications` MUST reject applications to internships where `deadline IS NOT NULL AND deadline < CURDATE()`, returning 400 "The application deadline for this internship has passed."

Canonical public visibility filter (use this exact WHERE everywhere):

```
-- For student/public-facing queries (listings, search, featured, recommendations, stats):
WHERE internship.status = 'active'
  AND employer_user.is_active = TRUE
  AND (internship.deadline IS NULL OR internship.deadline >= CURDATE())
```

This is the ONLY correct filter for public/student visibility. Do not ad-hoc construct visibility queries — always use this canonical filter or a shared query builder function that applies it.

Table: `skill`

Column	Type	Constraints	Notes
skill_id	INT	PK, AUTO_INCREMENT	

Column	Type	Constraints	Notes
display_name	VARCHAR(100)	NOT NULL	Human-readable form shown in UI (e.g., "JavaScript", "Node.js", "C++")
normalized_name	VARCHAR(100)	NOT NULL, UNIQUE	Canonical lowercase key for dedup (e.g., "javascript", "nodejs", "cpp")
category	ENUM('programming','web','data','ai_ml','devops','mobile','design','soft_skill','other')	NOT NULL, DEFAULT 'other'	Standardized categories matching AI prompt output

Normalization rule: Do NOT use a blanket "strip all special characters" approach — that would collapse `C++`, `C#`, and `C` into the same normalized name. Instead, use a **canonical alias map** for known special cases, then apply safe generic rules for everything else.

Normalization function (pseudocode):

```
CANONICAL_ALIASES = {
  "c++": "cpp",
  "c#": "csharp",
  "f#": "fsharp",
  ".net": "dotnet",
  "asp.net": "aspnet",
  "node.js": "nodejs",
  "express.js": "expressjs",
  "next.js": "nextjs",
  "vue.js": "vuejs",
  "react.js": "reactjs",
  "react native": "reactnative",
  "react-native": "reactnative",
  "three.js": "threejs",
  "d3.js": "d3js",
  "socket.io": "socketio",
  "scikit-learn": "scikitlearn",
  "objective-c": "objectivec",
  "t-sql": "tsql",
  "pl/sql": "plsql",
  "ci/cd": "cicd",
  "ui/ux": "uiux",
}

FUNCTION normalizeSkillName(input):
    lowered = LOWERCASE(TRIM(input))

    // Step 1: Check canonical alias map first (handles C++, C#, F#, .NET, Node.js, etc.)
    IF lowered IN CANONICAL_ALIASES:
        RETURN CANONICAL_ALIASES[lowered]

    // Step 2: Safe generic normalization for everything NOT in alias map:
    //   - lowercase (already done)
    //   - remove spaces, dots, hyphens, underscores, slashes
    //   - PRESERVE + and # (these are meaningful in language names like C++, C#)
    //     but since C++/C#/F# are already handled by the alias map above,
    //     any remaining + or # in unknown skills is safely preserved
    result = REGEX_REPLACE(lowered, /[\s.\-_\/]/g, "")

    RETURN result
```

Examples showing why this matters:

User types...	display_name stored	normalized_name stored	Duplicate?
"JavaScript"	JavaScript	javascript	First insert
"javascript"	JavaScript	javascript	Blocked
"Node.js"	Node.js	nodejs	First insert (alias map)
"NodeJS"	Node.js	nodejs	Blocked
"C++"	C++	cpp	First insert (alias map)
"c++"	C++	cpp	Blocked
"C#"	C#	csharp	First insert (alias map)
"c#"	C#	csharp	Blocked
"C"	C	c	First insert — correctly NOT collapsed with C++ or C#
"F#"	F#	fsharp	First insert (alias map)
".NET"	.NET	dotnet	First insert (alias map)

"React Native"	React Native	reactnative	First insert (alias map)
"react-native"	React Native	reactnative	Blocked (alias map)
"Machine Learning"	Machine Learning	machinelearning	First insert (generic rule)
"machine learning"	Machine Learning	machinelearning	Blocked
"machine-learning"	Machine Learning	machinelearning	Blocked (generic strips hyphens)
"Tailwind CSS"	Tailwind CSS	tailwindcss	First insert (generic rule)
"Tailwind-CSS"	Tailwind CSS	tailwindcss	Blocked (generic strips hyphens)
"tailwind/css"	Tailwind CSS	tailwindcss	Blocked (generic strips slashes)
"Tailwind_CSS"	Tailwind CSS	tailwindcss	Blocked (generic strips underscores)

The alias map is stored as a JSON config file (`utils/skillAliases.json`) and loaded once at server startup. It can be extended without code changes.

Indexes: `UNIQUE(normalized_name)` , `INDEX(category)`

Table: `has_skill` (student ↔ skill, many-to-many)

Column	Type	Constraints	Notes
student_user_id	INT	PK, FK → student.user_id ON DELETE CASCADE	
skill_id	INT	PK, FK → skill.skill_id ON DELETE CASCADE	
proficiency_level	ENUM('beginner','intermediate','advanced')	NOT NULL, DEFAULT 'intermediate'	
source	ENUM('manual','extracted')	NOT NULL, DEFAULT 'manual'	How the skill was added

Table: `requires_skill` (internship ↔ skill, many-to-many)

Column	Type	Constraints	Notes
internship_id	INT	PK, FK → internship.internship_id ON DELETE CASCADE	
skill_id	INT	PK, FK → skill.skill_id ON DELETE CASCADE	
required_level	ENUM('beginner','intermediate','advanced')	NOT NULL, DEFAULT 'intermediate'	
is_mandatory	BOOLEAN	NOT NULL, DEFAULT TRUE	Mandatory vs. nice-to-have

Table: `resume`

Column	Type	Constraints	Notes
resume_id	INT	PK, AUTO_INCREMENT	
student_user_id	INT	NOT NULL, FK → student.user_id ON DELETE CASCADE	
file_path	VARCHAR(255)	NOT NULL	Server path to stored file
original_filename	VARCHAR(255)	NOT NULL	Original uploaded name

Column	Type	Constraints	Notes
file_type	ENUM('pdf','docx')	NOT NULL	
extracted_text	LONGTEXT	NULLABLE	Raw text extracted from resume
created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	

Single-resume model: Each student has at most ONE resume at any time. Uploading a new resume replaces the existing one (old DB row is deleted, new row is inserted, old physical file is deleted ONLY if no `application.submitted_resume_path` references it — see "Resume file retention rule" above). The `student.primary_resume_id` always points to the student's single resume (or NULL if none). There is no resume selection dropdown — the student's one resume is always used. A `UNIQUE(student_user_id)` constraint enforces this at the DB level.

Indexes: `UNIQUE(resume_id, student_user_id)` (composite, enables the composite FK from `student.primary_resume_id` and `application.resume_id`), `UNIQUE(student_user_id)` (enforces single-resume model and also serves as the index for student lookups — no separate `INDEX(student_user_id)` needed).

Table: `application`

Column	Type	Constraints
application_id	INT	PK, AUTO_INCREMENT
student_user_id	INT	NOT NULL, FK → student.user_id ON DELETE CASCADE
internship_id	INT	NOT NULL, FK → internship.internship_id ON DELETE CASCADE
resume_id	INT	NULLABLE, composite FK (resume_id, student_user_id) → resume(resume_id, student_user_id) ON DELETE RESTRICT
submitted_resume_path	VARCHAR(255)	NOT NULL

Column	Type	Constraints
submitted_resume_filename	VARCHAR(255)	NOT NULL
cover_letter	TEXT	NULLABLE
status	ENUM('pending','under_review','interview_scheduled','accepted','rejected','withdrawn')	NOT NULL, DEFAULT 'pending'
match_score	DECIMAL(5,2)	NULLABLE
applied_date	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP
status_updated_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP
employer_note	TEXT	NULLABLE
updated_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP

Constraints: `UNIQUE(student_user_id, internship_id)` — one application per student per internship

Application Status State Machine (valid transitions):

```
pending → under_review (employer)
pending → rejected (employer)
pending → withdrawn (student)
under_review → interview_scheduled (employer)
under_review → accepted (employer)
under_review → rejected (employer)
under_review → withdrawn (student)
interview_scheduled → accepted (employer)
interview_scheduled → rejected (employer)
interview_scheduled → withdrawn (student)
```

Terminal states: `accepted`, `rejected`, `withdrawn` — no further transitions allowed. An employer cannot un-reject or un-accept; a student cannot un-withdraw. If a mistake is made, the employer should communicate with the student outside the status system (via messaging).

Backend enforcement: The `PUT /applications/:id/status` endpoint MUST validate the transition is in the allowed list above. Return 400 "Invalid status transition from '{old}' to '{new}'" if not allowed. The `PUT /applications/:id/withdraw` endpoint MUST validate the current status is `pending`, `under_review`, or `interview_scheduled`. Return 400 "Cannot withdraw an application that is already {status}" otherwise.

Duplicate application handling: If a student tries to apply to the same internship twice, the DB `UNIQUE(student_user_id, internship_id)` constraint catches it. The backend MUST catch this duplicate key error and return 409 "You have already applied to this internship" instead of a 500.

Indexes: `UNIQUE(student_user_id, internship_id)`, `INDEX(resume_id, student_user_id)` (composite FK), `INDEX(student_user_id, status)`, `INDEX(internship_id, status)`, `INDEX(applied_date)`

Table: `application_status_history` — NEW

Column	Type	Constraints	Note
history_id	INT	PK, AUTO_INCREMENT	
application_id	INT	NOT NULL, FK → application.application_id ON DELETE CASCADE	
old_status	ENUM('pending','under_review','interview_scheduled','accepted','rejected','withdrawn')	NULLABLE	NUL the i "pen entry
new_status	ENUM('pending','under_review','interview_scheduled','accepted','rejected','withdrawn')	NOT NULL	
changed_by_user_id	INT	NULLABLE, FK → users.user_id ON DELETE SET NULL	Who made chan (emp stude for witho or sy

Column	Type	Constraints	Note
note	TEXT	NULLABLE	Optio note expla the chan
created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	

Usage: On application creation, INSERT the first row with `old_status = NULL, new_status = 'pending', changed_by = student`. On every subsequent status change, INSERT a new row AND update `application.status` + `application.status_updated_at`. This gives a full audit trail from the moment of application: when was it submitted? When did it move to "under_review"? When was the interview scheduled? Who rejected it and why?

Indexes: `INDEX(changed_by_user_id)`, `INDEX(application_id, created_at)`

Table: `bookmark`

Column	Type	Constraints	Notes
student_user_id	INT	PK, FK → student.user_id ON DELETE CASCADE	
internship_id	INT	PK, FK → internship.internship_id ON DELETE CASCADE	
created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	

Table: `conversation`

Column	Type	Constraints	Notes
conversation_id	INT	PK, AUTO_INCREMENT	
student_user_id	INT	NOT NULL, FK → student.user_id ON DELETE CASCADE	
employer_user_id	INT	NOT NULL, FK → employer.user_id ON DELETE CASCADE	
internship_id	INT	NULLABLE, FK → internship.internship_id ON DELETE SET NULL	Live FK for lookups — becomes NULL when internship is deleted
context_type	ENUM('general','internship')	NOT NULL	Set at creation time, never changes
context_key	INT	NOT NULL	Immutable identifier: `0` for general conversations, original `internship_id` for internship-linked conversations. Never changes, even if internship is later deleted.
created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	

Uniqueness enforcement: `UNIQUE(student_user_id, employer_user_id, context_type, context_key)` — enforced at the DB level. This is stable because `context_key` is set once at creation and never modified, even when `internship_id` is nulled by deletion.

Why this design:
The previous approach used `COALESCE(internship_id, 0)` as a generated column for uniqueness. That breaks when two different internship-linked conversations both lose their `internship_id` via `ON DELETE SET NULL` — both collapse to key `0`, violating the unique constraint. The `context_key` approach preserves the original internship identity permanently.

Example:
conversation	student	employer	internship_id	context_type	context_key	What happens when internship 5 is deleted
General chat	101	201	NULL	general	0	Unaffected
About internship 5	101	201	5	internship	5	`internship_id` → NULL, but `context_key` stays 5. Uniqueness preserved.
About internship 9	101	201	9	internship	9	Unaffected

On creation: When creating a conversation, set `context_type` and `context_key` immutably:

```
IF internship_id IS PROVIDED:
    context_type = 'internship'
    context_key = internship_id
ELSE:
    context_type = 'general'
    context_key = 0
```

DB-level CHECK constraint enforces the following invariants:

```
CHECK (
  (context_type = 'general' AND context_key = 0 AND internship_id IS NULL) OR
  (context_type = 'internship' AND context_key > 0 AND (internship_id IS NULL OR internship_id = con
text_key))
)
```

This prevents:
- A general conversation with a non-zero `context_key` or a non-null `internship_id`
- An internship conversation with `context_key = 0`
- An internship conversation where `internship_id` exists but doesn't match `context_key` (data corruption)

The `internship_id IS NULL` branch in the internship case is necessary because `ON DELETE SET NULL` nulls `internship_id` when an internship is deleted — the CHECK must still pass after that happens.

Indexes: `INDEX(student_user_id)`, `INDEX(employer_user_id)`, `INDEX(internship_id)`, `UNIQUE(student_user_id, employer_user_id, context_type, context_key)`

Table: `message`

Column	Type	Constraints	Notes
message_id	INT	PK, AUTO_INCREMENT	

Column	Type	Constraints	Notes
conversation_id	INT	NOT NULL, FK → conversation.conversation_id ON DELETE CASCADE	
sender_user_id	INT	NULLABLE, FK → users.user_id ON DELETE SET NULL	Nullable for defensive integrity. In normal deletion flows, conversations cascade-delete their messages, so this SET NULL rarely triggers. May be NULL only in unusual scenarios where a message row survives its sender's account removal.
content	TEXT	NOT NULL	Message body
is_read	BOOLEAN	NOT NULL, DEFAULT FALSE	
created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	

Indexes: `INDEX(sender_user_id)`, `INDEX(conversation_id, created_at)`, `INDEX(conversation_id, is_read)`

Table: `notification`

Column	Type	Constraints	Notes
notification_id	INT	PK, AUTO_INCREMENT	
user_id	INT	NOT NULL, FK → users.user_id ON DELETE CASCADE	Recipient
type	VARCHAR(50)	NOT NULL	**Allowed values:** `application_received`, `application_status_changed`, `new_message`, `internship_approved`, `internship_rejected`, `invitation_received`, `welcome`, `new_user_registered`, `internship_pending_review`, `user_deactivated`. Must use only these values — enforce in backend constants. The last three are admin-specific: `new_user_registered` fires when a student or employer registers, `internship_pending_review` fires when an employer submits or resubmits an internship for approval, and `user_deactivated` fires when an admin deactivates a user account.
title	VARCHAR(200)	NOT NULL	
message	TEXT	NOT NULL	
reference_id	INT	NULLABLE	ID of related entity (polymorphic — acceptable pattern)
reference_type	VARCHAR(50)	NULLABLE	**Allowed values:** `application`, `internship`, `message`, `invitation`, `user`. Must match the entity type that `reference_id` points to. The `user` type is used for admin notifications that reference a user record (e.g., new registration, account deactivation).
is_read	BOOLEAN	NOT NULL, DEFAULT FALSE	

Column	Type	Constraints	Notes
email_sent	BOOLEAN	NOT NULL, DEFAULT FALSE	Whether email notification was also sent
created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	

Indexes: `INDEX(user_id, is_read, created_at)` — MySQL's B-tree index supports reverse scans, so explicit DESC is unnecessary for `ORDER BY created_at DESC` queries.

Table: `notification_preference` — NEW

Column	Type	Constraints	Notes
user_id	INT	PK, FK → users.user_id ON DELETE CASCADE	
email_application_status	BOOLEAN	NOT NULL, DEFAULT TRUE	**Student only.** Notify on application status changes
email_new_application	BOOLEAN	NOT NULL, DEFAULT TRUE	**Employer only.** Notify on new applications received
email_new_message	BOOLEAN	NOT NULL, DEFAULT TRUE	**Student and Employer.** Notify on new chat messages (admins do not have messaging and do not have preference rows — this field is irrelevant for admins)
email_recommendations	BOOLEAN	NOT NULL, DEFAULT TRUE	**Student only.** Notify on new recommended internships
email_internship_approved	BOOLEAN	NOT NULL, DEFAULT TRUE	**Employer only.** Notify when internship is approved/rejected
email_invitation	BOOLEAN	NOT NULL, DEFAULT TRUE	**Student only.** Notify when invited to apply
updated_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	

Auto-created with defaults when a student or employer registers. **Not created for admins** — admin notifications are always delivered with no opt-out.

Role-based preference visibility matrix:

Preference	Student UI	Employer UI	Admin UI
email_application_status	✓	—	—
email_new_application	—	✓	—
email_new_message	✓	✓	—
email_recommendations	✓	—	—
email_internship_approved	—	✓	—
email_invitation	✓	—	—

Admin does not have notification preferences — admin notifications are always delivered. The settings UI for each role shows ONLY the toggles marked ✓ for that role. All six booleans exist in the same DB row for simplicity, but unused ones for a given role stay at their default (TRUE) and are ignored.

Table: `internship_invitation` — NEW

Column	Type	Constraints	Notes
invitation_id	INT	PK, AUTO_INCREMENT	
internship_id	INT	NOT NULL	
student_user_id	INT	NOT NULL, FK → student.user_id ON DELETE CASCADE	Invited student
employer_user_id	INT	NOT NULL	Who sent invitation
message	TEXT	NULLABLE	Optional personal message from employer
status	ENUM('pending','viewed','applied','dismissed')	NOT NULL, DEFAULT 'pending'	
created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	

Ownership constraint: Composite FK `(internship_id, employer_user_id) → internship(internship_id, employer_user_id) ON DELETE CASCADE`. This enforces at the DB level that only the employer who owns the internship can create an invitation for it. Replaces separate individual FKs on internship_id and employer_user_id.

Constraints: `UNIQUE(internship_id, student_user_id)` — prevent duplicate invitations

Invitation Lifecycle:
- **On creation (`POST /employer/internships/:internshipId/invite/:studentId`):** Creates invitation with status `pending`. Sends notification + email to student. Only allowed if internship is `active` and no existing invitation for this student+internship pair.
- **On student viewing an invitation:** Student explicitly marks an invitation as viewed via `PUT /student/invitations/:id/view`. This sets status from `pending` to `viewed`. Do NOT auto-mark on list fetch — that would trigger on preloads and polling, giving false "viewed" signals.
- **On student applying to the invited internship:** The `POST /applications` endpoint checks for an existing invitation matching this student+internship pair. If found, updates invitation status to `applied`. The student can apply regardless of whether an invitation exists — the invitation is informational, not a prerequisite.
- **On student dismissing:** `PUT /student/invitations/:id/dismiss` sets status to `dismissed`. The student can still apply later even after dismissing.
- **Student-side routes:**
- `GET /student/invitations` — list all invitations for this student (paginated, sorted by created_at DESC)
- `PUT /student/invitations/:id/view` — mark an invitation as viewed (only if current status is `pending`)
- `PUT /student/invitations/:id/dismiss` — dismiss an invitation

Indexes: `UNIQUE(internship_id, student_user_id)`, `INDEX(employer_user_id)`, `INDEX(student_user_id, status)`, `INDEX(internship_id)`, `INDEX(internship_id, employer_user_id)` (composite FK for ownership)

Table: `internship_view` — NEW (enables time-series analytics)

Column	Type	Constraints	Notes
view_id	INT	PK, AUTO_INCREMENT	
internship_id	INT	NOT NULL, FK → internship.internship_id ON DELETE CASCADE	
viewer_user_id	INT	NULLABLE, FK → users.user_id ON DELETE SET NULL	The logged-in student who viewed. Views are only inserted for authenticated students — this is never NULL on insert. Becomes NULL only if the user account is later deleted (ON DELETE SET NULL preserves the view record for analytics while anonymizing it).
viewed_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	

Usage: Views are logged ONLY for logged-in students viewing active internships (not for anonymous visitors, not for the owning employer viewing their own listing, not for admins). Total views = `COUNT(*)` . Weekly chart = `GROUP BY YEARWEEK(viewed_at)`. Deduplicate by checking for existing `(internship_id, viewer_user_id)` within the last hour before INSERT.

Indexes:
- `INDEX(internship_id, viewed_at)` — for total count and weekly aggregation
- `INDEX(internship_id, viewer_user_id, viewed_at)` — for dedupe check

Table: `password_reset_token`

Column	Type	Constraints	Notes
token_id	INT	PK, AUTO_INCREMENT	
user_id	INT	NOT NULL, FK → users.user_id ON DELETE CASCADE	
token_hash	CHAR(64)	NOT NULL, UNIQUE	SHA-256 hex hash of the raw token (raw token is sent via email, never stored). Fixed 64 chars = exact SHA-256 hex length.
expires_at	DATETIME	NOT NULL	Token expiry (1 hour from creation)
used	BOOLEAN	NOT NULL, DEFAULT FALSE	
created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	

Indexes: `INDEX(user_id, expires_at, used)` — the `token_hash` column already has a UNIQUE constraint which creates an implicit index, so no additional INDEX on `token_hash` is needed.

Table: `system_log`

Column	Type	Constraints	Notes
log_id	INT	PK, AUTO_INCREMENT	

Column	Type	Constraints	Notes
user_id	INT	NULLABLE, FK → users.user_id ON DELETE SET NULL	NULL for system-generated events
action	VARCHAR(200)	NOT NULL	**Allowed values:** `user_registered`, `user_login`, `user_logout`, `user_deactivated`, `user_activated`, `user_deleted`, `password_changed`, `password_reset_requested`, `password_reset_completed`, `internship_posted`, `internship_approved`, `internship_rejected`, `internship_closed`, `internship_reopened`, `internship_deleted`, `application_submitted`, `application_status_changed`, `application_withdrawn`, `resume_uploaded`, `resume_deleted`, `profile_updated`, `admin_action`. Enforce in backend constants.
details	TEXT	NULLABLE	Additional context
ip_address	VARCHAR(45)	NULLABLE	
created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	

Indexes: `INDEX(user_id)`, `INDEX(action)`, `INDEX(created_at)`

Table: `report`

Column	Type	Constraints	Notes
report_id	INT	PK, AUTO_INCREMENT	
admin_user_id	INT	NULLABLE, FK → admin.user_id ON DELETE SET NULL	Who generated it — NULL if admin account removed, log preserved
report_type	VARCHAR(100)	NOT NULL	**Allowed values:** `user_activity`, `internship_applications`, `student_match_ranking`, `employer_performance`, `system_audit`. Enforce in backend constants.
report_description	TEXT	NULLABLE	
filters_json	TEXT	NULLABLE	JSON string of the filters/date range used to generate this report, for audit purposes
created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	

Note: Reports are generated on demand — no output files are stored on disk. This table serves as an audit log of which reports were requested, by whom, with what filters, and when. The actual report data is re-queried from the database each time.

Table: `internship_embedding`

Column	Type	Constraints	Notes
internship_id	INT	PK, FK → internship.internship_id ON DELETE CASCADE	
embedding	JSON	NOT NULL	768-dim vector as JSON array of floats
source_text_hash	VARCHAR(64)	NOT NULL	SHA-256 of source text — detect when re-embedding needed
model_name	VARCHAR(100)	NOT NULL, DEFAULT 'text-embedding-004'	Track which model generated this embedding
dimensions	INT	NOT NULL, DEFAULT 768	Embedding vector size
created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	
updated_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	

Table: `student_embedding`

Column	Type	Constraints	Notes
student_user_id	INT	PK, FK → student.user_id ON DELETE CASCADE	
embedding	JSON	NOT NULL	768-dim vector as JSON array of floats
source_text_hash	VARCHAR(64)	NOT NULL	
model_name	VARCHAR(100)	NOT NULL, DEFAULT 'text-embedding-004'	
dimensions	INT	NOT NULL, DEFAULT 768	
created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	
updated_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	

Seed Data: Skills Dictionary

Pre-populate the `skill` table with 150+ skills across categories:
- **Programming Languages** (30+): JavaScript, Python, Java, C++, C#, TypeScript, PHP, Ruby, Swift, Kotlin, Go, Rust, R, MATLAB, Scala, Perl, Dart, Lua, Shell/Bash, Assembly, Objective-C, Haskell, Elixir, Clojure, F#, COBOL, Fortran, SQL, NoSQL, GraphQL
- **Web Development** (25+): React, Angular, Vue.js, Next.js, Node.js, Express.js, Django, Flask, Spring Boot, ASP.NET, HTML, CSS, Tailwind CSS, Bootstrap, SASS, jQuery, WordPress, Laravel, Ruby on Rails, Svelte, Gatsby, REST API, GraphQL API, WebSocket, OAuth
- **Data & Databases** (20+): MySQL, PostgreSQL, MongoDB, Firebase, Redis, SQLite, Oracle, SQL Server, Elasticsearch, DynamoDB, Cassandra, Data Analysis, Data Visualization, Pandas, NumPy, Tableau, Power BI, ETL, Data Warehousing, Big Data
- **AI & Machine Learning** (15+): Machine Learning, Deep Learning, TensorFlow, PyTorch, scikit-learn, NLP, Computer Vision, Neural Networks, Reinforcement Learning, OpenCV, Keras, Hugging Face, LLM, Prompt

Engineering, Data Science
- **DevOps & Cloud** (20+): AWS, Azure, Google Cloud, Docker, Kubernetes, CI/CD, Jenkins, GitHub Actions, Terraform, Ansible, Linux, Nginx, Apache, Git, GitHub, GitLab, Heroku, Vercel, Netlify, Serverless
- **Mobile Development** (10+): React Native, Flutter, Swift, Kotlin, Android, iOS, Xamarin, Ionic, Expo, Mobile UI
- **Design & Tools** (15+): Figma, Adobe XD, Photoshop, Illustrator, Canva, UI Design, UX Design, Wireframing, Prototyping, User Research, Accessibility, Responsive Design, Design Systems, Sketch, InVision
- **Soft Skills** (15+): Communication, Teamwork, Leadership, Problem Solving, Critical Thinking, Project Management, Agile, Scrum, Time Management, Presentation, Writing, Research, Adaptability, Creativity, Attention to Detail

---

AI & NLP ENGINE — FULL SPECIFICATION

The AI engine is the core of InternMatch. It consists of **three distinct NLP/AI subsystems**, each using real AI techniques — not keyword matching.

SUBSYSTEM 1: LLM-Powered Skill Extraction from Resumes

Technology: Google Gemini 2.5 Flash-Lite API (free tier)
Technique: Large Language Model (LLM) prompt-based Named Entity Recognition (NER) for skill extraction

How It Works:

1. Student uploads a resume (PDF or DOCX)
2. Backend extracts raw text using `pdf-parse` or `mammoth`
3. Extracted text is sent to the Gemini API with a structured prompt
4. Gemini returns a JSON array of identified skills with estimated proficiency levels and categories
5. Backend cross-references extracted skills against the `skill` table — matching existing skills and flagging new ones
6. Student is shown the results to review, edit, accept, or reject before saving

The Prompt (sent to Gemini):

```
You are an expert HR recruiter and technical skill analyst. Analyze the following resume text and
extract all professional skills, technical skills, tools, programming languages, frameworks, soft
skills, and competencies.

For each skill found, provide:
1. skill_name: the normalized, commonly-used name of the skill (e.g., "JavaScript" not "JS",
"Machine Learning" not "ML")
2. category: one of ["programming", "web", "data", "ai_ml", "devops", "mobile", "design",
"soft_skill", "other"]
3. proficiency_estimate: estimate based on context clues in the resume — "beginner", "intermediate",
or "advanced". Look for signals like:
   - "beginner": mentioned in coursework or listed without elaboration
   - "intermediate": used in projects, 1-2 years experience mentioned
   - "advanced": 3+ years, led projects with it, certifications, in-depth usage described

Return ONLY a valid JSON array with no additional text, no markdown, no explanation. Example format:
[
  {"skill_name": "Python", "category": "programming", "proficiency_estimate": "advanced"},
  {"skill_name": "React", "category": "web", "proficiency_estimate": "intermediate"}
]

Resume text:
---
{resumeText}
---
```

Backend Processing After LLM Response:

```
FUNCTION processLLMSkillExtraction(llmResponse, studentId):
    parsedSkills = JSON.parse(llmResponse)
    results = []

    FOR EACH extractedSkill IN parsedSkills:
        // Normalize the skill name for dedup lookup
        normalizedName = normalizeSkillName(extractedSkill.skill_name)  // uses canonical alias map
+ safe generic rules
        existingSkill = SELECT * FROM skill WHERE normalized_name = normalizedName

        IF existingSkill NOT FOUND:
            // DO NOT insert into the canonical skill table yet.
            // Mark as a new/unknown skill and return it for review with skill_id = null.
            // The skill will only be inserted into the skill table when the student
            // confirms it during the review step. This prevents LLM hallucinations,
            // typos, and junk from polluting the global skill dictionary.
            skillId = null
            skillName = extractedSkill.skill_name
        ELSE:
            skillId = existingSkill.skill_id
            skillName = existingSkill.display_name

        results.APPEND({
            skill_id: skillId,                // null for new/unknown skills
            skill_name: skillName,
            category: extractedSkill.category,
            proficiency_level: extractedSkill.proficiency_estimate,
            source: 'extracted',
            is_new: skillId == null            // flag for frontend to show "new skill" indicator
        })

    // Return to frontend for student review — DO NOT auto-save
    RETURN results
```

Skill confirmation step (when student accepts extracted skills):
When the student reviews the extracted skills and clicks "Confirm" (or selects individual skills to accept), the frontend sends the accepted list to `POST /api/student/skills`. For each accepted skill:
- If `skill_id` is not null → the skill already exists in the dictionary. Insert into `has_skill` normally.
- If `skill_id` is null (new skill) → NOW insert into the canonical `skill` table with proper normalization, then insert into `has_skill`. This ensures only human-confirmed skills enter the global dictionary.
- Skills the student deselected or removed during review are simply not sent — they are discarded.

Why This is Real AI:

- Uses a **Large Language Model** (Gemini) for natural language understanding
- The LLM understands context: "Built a REST API in Node.js for 2 years" → extracts "Node.js" AND "REST API" with "intermediate"/"advanced" proficiency based on the "2 years" context
- It catches skills that keyword matching would miss: "developed responsive layouts" → "Responsive Design", "CSS"
- It avoids false positives that keyword matching causes: "I am not experienced in Java" → keyword matching would extract "Java", the LLM will NOT

- **Bidirectional extraction:** the same NER technique is applied to both student resumes (extracting what they know) AND internship descriptions (extracting what's required), ensuring both sides of the matching equation have accurate, AI-identified skill data
- For internship descriptions, the LLM also classifies mandatory vs. optional from language cues: "must have React" → mandatory, "Docker is a plus" → optional — something keyword matching cannot do
- This is a form of **zero-shot NER (Named Entity Recognition)** — an established NLP technique

Fallback Strategy:

If the Gemini API is unavailable (rate limit, outage), fall back to a local regex-based keyword extraction against the skills dictionary. This ensures the system never fully breaks. The fallback is clearly marked as "basic extraction" in the UI.

LLM response parsing: The Gemini response MUST be parsed safely:
1. Try `JSON.parse(llmResponse)` directly.
2. If that fails, try to extract the first JSON array from the response: `const match = llmResponse.match(/\[[\s\S]*\]/); JSON.parse(match[0])` (LLMs sometimes wrap JSON in markdown fences or add preamble text).
3. If that also fails, log the raw response for debugging and fall back to the regex keyword extraction.
4. After successful parsing, validate the structure: each item must have `skill_name` (string), `category` (valid enum value), and `proficiency_estimate` (valid enum value). Skip any items that fail validation rather than rejecting the entire batch.

SUBSYSTEM 1B: LLM-Powered Skill Extraction from Internship Descriptions

Technology: Same as Subsystem 1 — Google Gemini 2.5 Flash-Lite API
Technique: LLM prompt-based extraction of required skills, proficiency levels, and mandatory/optional classification from job description text

How It Works:

1. Employer writes the internship description in the Post/Edit Internship form
2. Employer clicks the **"Analyze Description"** button (separate from the "Post Internship" button)
3. Backend sends the description text to the Gemini API with a structured prompt
4. Gemini returns a JSON array of identified skills with estimated required proficiency levels, categories, and mandatory/optional classification
5. Backend cross-references extracted skills against the `skill` table — matching existing skills and flagging new ones
6. Employer is shown the results in the Required Skills section to review, edit, accept, or reject before saving
7. Extracted skills are **merged** with any skills the employer already added manually — duplicates are deduplicated by normalized_name, with the employer's manual entry taking priority if both exist

The Prompt (sent to Gemini):

```
You are an expert HR recruiter and job requirements analyst. Analyze the following internship/job
description and extract all required professional skills, technical skills, tools, programming
languages, frameworks, soft skills, and competencies.

For each skill found, provide:
1. skill_name: the normalized, commonly-used name of the skill (e.g., "JavaScript" not "JS",
"Machine Learning" not "ML")
2. category: one of ["programming", "web", "data", "ai_ml", "devops", "mobile", "design",
"soft_skill", "other"]
3. required_level: estimate the required proficiency level based on context clues — "beginner",
"intermediate", or "advanced". Look for signals like:
   - "beginner": "basic understanding", "familiarity with", "exposure to", "coursework in"
   - "intermediate": "experience with", "proficient in", "1-2 years", "working knowledge"
   - "advanced": "expert in", "3+ years", "deep experience", "lead", "architect", "senior-level"
4. is_mandatory: determine if this skill is mandatory or optional based on language cues:
   - mandatory (true): "required", "must have", "essential", "mandatory", "need", "should have",
listed under "Requirements" or "Qualifications" sections, or stated without any optional qualifier
   - optional (false): "nice to have", "preferred", "bonus", "plus", "ideal", "desired", listed
under "Preferred" or "Nice to Have" sections

Return ONLY a valid JSON array with no additional text, no markdown, no explanation. Example format:
[
  {"skill_name": "React", "category": "web", "required_level": "intermediate", "is_mandatory":
true},
  {"skill_name": "Docker", "category": "devops", "required_level": "beginner", "is_mandatory":
false},
  {"skill_name": "Communication", "category": "soft_skill", "required_level": "intermediate",
"is_mandatory": true}
]

Internship description:
---
{descriptionText}
---
```

Backend Processing After LLM Response:

```
FUNCTION processInternshipSkillExtraction(llmResponse, internshipId):
    parsedSkills = JSON.parse(llmResponse)  // with same safe parsing as resume extraction
    results = []

    FOR EACH extractedSkill IN parsedSkills:
        normalizedName = normalizeSkillName(extractedSkill.skill_name)
        existingSkill = SELECT * FROM skill WHERE normalized_name = normalizedName

        IF existingSkill NOT FOUND:
            // DO NOT insert into canonical skill table yet.
            // Same rule as resume extraction: only insert after employer confirms.
            skillId = null
            skillName = extractedSkill.skill_name
        ELSE:
            skillId = existingSkill.skill_id
            skillName = existingSkill.display_name

        results.APPEND({
            skill_id: skillId,              // null for new/unknown skills
            skill_name: skillName,
            category: extractedSkill.category,
            required_level: extractedSkill.required_level,
            is_mandatory: extractedSkill.is_mandatory,
            source: 'extracted',
            is_new: skillId == null
        })

    // Return to frontend for employer review — DO NOT auto-save
    RETURN results
```

Skill confirmation (on internship submit/save): When the employer saves the internship, any accepted extracted skills with `skill_id = null` are inserted into the canonical `skill` table at that point, then linked via `requires_skill`. Same deferred-insert pattern as resume extraction.

Merge Logic (extracted + manually added):

When the employer already has manually-added skills in the Required Skills section:
- Extracted skills that match an existing manual skill (by normalized_name) are **skipped** — the employer's manual entry takes priority for level and mandatory/optional classification
- Extracted skills that are new (not already in the list) are **appended** to the list with a visual indicator showing they were AI-suggested
- The employer can then review the merged list: edit levels, toggle mandatory/optional, remove any they don't want, and add more manually
- Only when the employer clicks "Post Internship" or "Save Changes" are the skills committed to the `requires_skill` table

When Extraction Runs:

- **Post Internship form:** When the employer clicks "Analyze Description" button. The button is enabled only when the description field has content (minimum 50 characters to produce meaningful results).
- **Edit Internship form:** When the employer edits the description and clicks "Re-analyze Description". The button appears only when the description text has changed from the saved version. Extracted results merge with the existing required skills list.

- **Never auto-triggered:** The extraction does NOT run automatically on form submission. The employer must explicitly click the analyze button.

Fallback Strategy:

Same as resume extraction — if Gemini is unavailable, the employer uses the manual skill picker only. The "Analyze Description" button shows an error toast: "AI analysis is temporarily unavailable. Please add skills manually." The manual skill picker always works regardless of API status.

Same Defensive Parsing:

Same safe parsing rules as Subsystem 1: try JSON.parse → regex for `[...]` → fallback. Validate each item has `skill_name`, `category`, `required_level` (valid enum), and `is_mandatory` (boolean). Skip invalid items.

SUBSYSTEM 2: Hybrid AI Matching Engine (Skill Matching + Semantic Similarity)

Technique: Weighted skill-based scoring COMBINED WITH semantic embedding similarity
This is a hybrid recommendation approach — combining content-based filtering (skill matching) with NLP-based semantic analysis (embedding similarity). This is academically stronger than either approach alone.

Architecture: Two-Score Fusion

```
Final Match Score = (Skill Match Score × 0.65) + (Semantic Similarity Score × 0.20) + (Profile Bonus
× 0.15)
```

Component 1 — Skill Match Score (65% weight): Structured comparison of student skills vs. internship required skills, accounting for mandatory/optional and proficiency levels.

Component 2 — Semantic Similarity Score (20% weight): Uses Gemini text embeddings to compute the semantic similarity between the student's full profile text and the internship's full description. This catches "soft" compatibility that skill tags alone miss.

Component 3 — Profile Bonus (15% weight): GPA, major relevance, and experience factors.

Component 1: Skill Match Score (detailed)

```
FUNCTION calculateSkillMatchScore(studentSkills, internshipRequiredSkills):
    mandatorySkills = internshipRequiredSkills WHERE is_mandatory = TRUE
    optionalSkills = internshipRequiredSkills WHERE is_mandatory = FALSE
    studentSkillMap = MAP(studentSkills: skill_id → proficiency_level)

    // Score mandatory skills
    mandatoryPoints = 0
    mandatoryMax = SIZE(mandatorySkills)
    FOR EACH reqSkill IN mandatorySkills:
        IF reqSkill.skill_id IN studentSkillMap:
            studentLevel = numericLevel(studentSkillMap[reqSkill.skill_id])  // beginner=1,
intermediate=2, advanced=3
            requiredLevel = numericLevel(reqSkill.required_level)
            IF studentLevel >= requiredLevel:
                mandatoryPoints += 1.0    // Full match
            ELSE IF studentLevel == requiredLevel - 1:
                mandatoryPoints += 0.6    // Close match (one level below)
            ELSE:
                mandatoryPoints += 0.3    // Weak match (two levels below)

    mandatoryScore = IF mandatoryMax > 0 THEN (mandatoryPoints / mandatoryMax) × 100 ELSE 100

    // Score optional skills (same point logic)
    optionalPoints = 0
    optionalMax = SIZE(optionalSkills)
    FOR EACH reqSkill IN optionalSkills:
        IF reqSkill.skill_id IN studentSkillMap:
            [same point logic as mandatory]

    optionalScore = IF optionalMax > 0 THEN (optionalPoints / optionalMax) × 100 ELSE 0

    // Weighted combination: mandatory matters more
    RETURN (mandatoryScore × 0.75) + (optionalScore × 0.25)
```

Component 2: Semantic Similarity Score (detailed)

This is the NLP part. It uses **text embeddings** — numerical vector representations of text that capture semantic meaning.

```
FUNCTION calculateSemanticSimilarity(student, internship):
    // Build student profile text
    studentText = CONCATENATE(
        "Major: " + student.major,
        "University: " + student.university,
        "Bio: " + student.bio,
        "Skills: " + JOIN(student.skills, ", "),
        "Resume summary: " + FIRST_500_WORDS(student.resume.extracted_text)
    )

    // Build internship text
    internshipText = CONCATENATE(
        "Title: " + internship.title,
        "Description: " + internship.description,
        "Industry: " + internship.employer.industry,
        "Required skills: " + JOIN(internship.requiredSkills, ", "),
        "Location: " + internship.location
    )

    // Generate embeddings using Gemini Embedding API
    studentEmbedding = geminiEmbed(studentText)      // Returns 768-dim vector
    internshipEmbedding = geminiEmbed(internshipText) // Returns 768-dim vector

    // Compute cosine similarity
    similarity = cosineSimilarity(studentEmbedding, internshipEmbedding)
    // Result is between -1 and 1; normalize to 0-100
    RETURN (similarity + 1) / 2 × 100
```

Cosine Similarity formula:

```
cosineSimilarity(A, B) = (A · B) / (||A|| × ||B||)

Where:
  A · B = sum of (A[i] × B[i]) for all dimensions
  ||A|| = sqrt(sum of A[i]²)
```

Embedding Caching Strategy:

- **Internship embeddings**: Generated when an internship is approved (including first-time approval AND re-approval after a rejection → edit → re-submit cycle). Stored in the `internship_embedding` table. Re-generated if the internship description, title, or required skills are edited while active (triggered by `PUT /employer/internships/:id`). The approval endpoint (`PUT /internships/:id/approve`) is the canonical trigger for first-time embedding generation.
- **Student embeddings**: Generated when a student updates their profile (bio, major, university, skills), OR uploads/replaces their resume. Stored in the `student_embedding` table. Since students have a single resume, any resume upload triggers: (1) re-extract skills from the resume via the LLM, (2) re-generate the student embedding vector. All cached match scores for this student become stale and are recomputed on next access.
- This means the Gemini API is called only on data changes, NOT on every recommendation request. This keeps API usage very low.

Component 3: Profile Bonus (detailed)

```
FUNCTION calculateProfileBonus(student, internship):
    gpaBonus = IF student.gpa IS NULL THEN 0  // GPA not provided — no bonus, no penalty
               ELSE IF student.gpa >= 3.5 THEN 30
               ELSE IF student.gpa >= 3.0 THEN 20
               ELSE IF student.gpa >= 2.5 THEN 10
               ELSE 0

    // Major relevance: determined by a static lookup table mapping majors to relevant industries.
    // Stored as a JSON config file (utils/majorIndustryMap.json) loaded once at server startup.
    // Example entries:
    //   "computer science": ["Technology", "Finance", "Engineering"],
    //   "marketing": ["Marketing", "Education"],
    //   "electrical engineering": ["Technology", "Engineering"],
    //   "finance": ["Finance", "Technology"],
    //   "graphic design": ["Marketing", "Education", "Technology"],
    //   "biology": ["Healthcare", "Education"],
    //   "business administration": ["Finance", "Marketing", "Technology", "Education",
"Engineering"]
    // Lookup is case-insensitive. If student.major (lowercased, trimmed) is found in the map
    // AND internship.employer.industry is in that major's industry list → relevant.
    // If the major is not in the map at all → default to NOT relevant (0 bonus).
    // The map can be extended without code changes by editing the JSON file.
    majorBonus = IF majorIsRelevant(student.major, internship.employer.industry) THEN 50 ELSE 0

    experienceBonus = IF student has any 'accepted' applications THEN 20 ELSE 0

    RETURN ((gpaBonus + majorBonus + experienceBonus) / 100) × 100  // Normalize to 0-100
```

Graduated Student Handling:

Students who have already graduated (`graduation_year < YEAR(CURDATE())` — computed on read, not stored) are handled as follows:

- **Can still use the platform fully:** browse internships, apply, message employers, get match scores. Graduation does NOT block any functionality.
- **Flagged in employer views:** When employers view applicants or AI candidate recommendations, graduated students show a badge: "Graduated ({year})" next to their university/major. This helps employers make informed decisions.
- **Not filtered out by default:** Graduated students appear in candidate recommendations and search results alongside enrolled students. Employers can optionally filter by enrollment status if a filter is provided.
- **Match score is unaffected:** Graduation status does not penalize or boost the match score. The matching algorithm evaluates skills, semantics, and profile data regardless of enrollment status.
- **Profile displays graduation status:** The student profile page shows "Graduated" or "Expected graduation: {year}" based on whether graduation_year has passed.

Full Combined Score:

```
FUNCTION calculateFinalMatchScore(student, internship):
    skillScore = calculateSkillMatchScore(student.skills, internship.requiredSkills)
    semanticScore = calculateSemanticSimilarity(student, internship)
    profileBonus = calculateProfileBonus(student, internship)

    finalScore = (skillScore × 0.65) + (semanticScore × 0.20) + (profileBonus × 0.15)
    RETURN ROUND(finalScore, 2)  // 0.00 to 100.00 — matches DECIMAL(5,2) storage in
application.match_score
```

Algorithm Edge Cases & Guarantees:

Edge case 1 — Student has zero skills:
- `calculateSkillMatchScore` receives empty `studentSkills` map.
- For mandatory skills: every required skill is unmatched → `mandatoryPoints = 0`, `mandatoryScore = 0`.
- For optional skills: same → `optionalScore = 0`.
- Result: `skillScore = 0`. The semantic and profile bonus components still contribute, so the final score is not necessarily zero.

Edge case 2 — Internship has zero required skills:
- `mandatoryMax = 0` → `mandatoryScore = 100` (default when no mandatory skills). `optionalMax = 0` → `optionalScore = 0`.
- Result: `skillScore = (100 × 0.75) + (0 × 0.25) = 75`. This is intentional — an internship with no skill requirements should give partial credit, not full credit, because the lack of requirements provides no signal of fit.

Edge case 3 — Student has no resume (no extracted text):
- `calculateSemanticSimilarity` builds `studentText` without resume summary: "Resume summary: " with empty string.
- Embedding is generated from available fields (major, university, bio, skills). Score will be lower fidelity but functional.
- The system should display a "Complete your profile for better matches" prompt.

Edge case 4 — Student has no GPA:
- `gpaBonus = 0` (not penalized — the bonus is opt-in). Students without GPA simply don't get that component of the profile bonus.

Edge case 5 — Internship has ONLY optional skills, no mandatory:
- `mandatoryMax = 0` → `mandatoryScore = 100`. `optionalMax > 0` → `optionalScore` calculated normally.
- Result: `skillScore = (100 × 0.75) + (optionalScore × 0.25)`. The optional skills still matter.

Edge case 6 — Student has skills but none match the internship at all:
- Both mandatory and optional points = 0. `skillScore = 0`. Semantic similarity may still find relevance if the student's bio/resume discusses related topics.

Edge case 7 — Embedding not yet generated (new student, new internship):
- If `student_embedding` row does not exist for the student, OR `internship_embedding` row does not exist for the internship: skip the semantic similarity component. Recalculate with adjusted weights: `finalScore = (skillScore × 0.80) + (profileBonus × 0.20)`. Log a warning. The missing embedding should be generated on next profile/internship access.

Edge case 8 — Gemini API fails during embedding generation:
- If the embedding API call fails (rate limit, network error, API outage): do NOT crash. Log the failure. Store no embedding (or keep the old one if it exists). The matching system falls back to adjusted weights as in edge case 7. Retry on next relevant trigger (profile update, page access).

Edge case 9 — Gemini API returns invalid JSON during skill extraction:
- Wrap `JSON.parse(llmResponse)` in try-catch. If parsing fails: attempt to extract JSON from the response by looking for `[...]` brackets (LLMs sometimes wrap JSON in markdown). If that also fails: fall back to the regex-based keyword extraction (fallback strategy). Mark the extraction as "basic extraction" in the UI. Log the raw response for debugging.

Edge case 10 — Student's profile text is too short for meaningful embedding:
- If the concatenated student profile text is under 20 characters (e.g., just "Major: CS"): generate the embedding anyway (the model handles short text), but flag the student's profile as "incomplete" and show a "Complete your profile for better match accuracy" warning in the UI.

Edge case 11 — Cosine similarity returns negative value:
- Rare but possible. The normalization `(similarity + 1) / 2 × 100` handles this: a cosine similarity of -1 maps to 0, and +1 maps to 100. No special handling needed.

Edge case 12 — Division by zero in skill scoring:
- Already handled: `IF mandatoryMax > 0 THEN ... ELSE 100` and `IF optionalMax > 0 THEN ... ELSE 0`. No division by zero is possible.

Why This is Real AI:

- **Text embeddings** are a core NLP technique used in modern search engines, recommendation systems, and AI applications
- **Cosine similarity on embeddings** is the same technique used by production systems like LinkedIn's job matching, Google's semantic search, and Netflix recommendations
- The **hybrid approach** (structured skill matching + semantic embedding) is a recognized pattern in recommendation system research — it's more robust than either approach alone
- The semantic component catches matches that pure skill comparison misses: e.g., a student with "data visualization" skills matches an internship asking for "Tableau and Power BI" even if the exact tool names don't match, because the embeddings understand they're semantically related

---

SUBSYSTEM 3: NLP-Powered Semantic Search

Technology: Gemini Embedding API (`text-embedding-004`)
Technique: Semantic vector search using cosine similarity between query embeddings and pre-computed internship embeddings

This replaces traditional SQL `LIKE '%keyword%'` search with intelligent semantic understanding.

How It Works:

1. When an internship is approved by admin, its full text (title + description + skills + location) is embedded using Gemini and stored as a 768-dimensional vector in the `internship_embedding` table
2. When a student types a search query (e.g., "I want to work on building mobile apps"), the query is embedded using the same model
3. Cosine similarity is computed between the query vector and ALL internship vectors

4. Results are ranked by similarity score and returned
5. Traditional filters (location, work type, duration, salary) are applied ON TOP of the semantic ranking

Why This Matters:

Student types...	Keyword search finds...	NLP semantic search finds...
"building websites"	Only results containing "building" AND "websites"	React, HTML/CSS, Frontend Developer, Web Development internships
"data stuff"	Almost nothing useful	Data Analysis, Data Science, SQL, Business Intelligence internships
"I like working with people"	Nothing relevant	Marketing, Sales, HR, Customer Relations internships
"AI and smart systems"	Only results with "AI" literally	Machine Learning, Deep Learning, Data Science, NLP internships
"making apps for phones"	Nothing	Mobile Development, React Native, Flutter, iOS/Android internships

Search Algorithm:

```
FUNCTION semanticSearch(queryText, filters):
    // Step 1: Generate embedding for search query
    queryEmbedding = geminiEmbed(queryText)  // 768-dim vector

    // Step 2: Load all internship embeddings matching the canonical public visibility filter
    internshipEmbeddings = SELECT internship_id, embedding FROM internship_embedding
                           JOIN internship ON ...
                           JOIN employer ON ... JOIN users ON ...
                           WHERE internship.status = 'active'
                             AND users.is_active = TRUE
                             AND (internship.deadline IS NULL OR internship.deadline >= CURDATE())

    // Step 3: Calculate similarity scores
    results = []
    FOR EACH intern IN internshipEmbeddings:
        similarity = cosineSimilarity(queryEmbedding, intern.embedding)
        results.APPEND({ internship_id: intern.internship_id, relevance_score: similarity })

    // Step 4: Sort by relevance
    results = SORT(results, BY relevance_score, DESCENDING)

    // Step 5: Apply traditional filters (SQL WHERE clauses)
    IF filters.location: results = results WHERE internship.location MATCHES filters.location
    IF filters.workType: results = results WHERE internship.work_type IN filters.workType
    IF filters.duration: results = results WHERE internship.duration_months IN filters.duration
    IF filters.paidOnly: results = results WHERE (internship.salary_min > 0 OR internship.salary_max
> 0)

    // Step 6: Combine with match score if student is logged in
    IF student IS authenticated:
        FOR EACH result IN results:
            result.match_score = calculateFinalMatchScore(student, result.internship)

    RETURN results (paginated, 10 per page)
```

Fallback for Short/Simple Queries:

If the search query is very short (< 3 words) AND matches a common pattern (location name, company name, exact skill name), supplement semantic search with a traditional SQL text search to ensure exact matches aren't missed:

```
FUNCTION hybridSearch(queryText, filters):
    semanticResults = semanticSearch(queryText, filters)
    keywordResults = SQL: SELECT * FROM internship WHERE title LIKE '%query%' OR description LIKE
'%query%' OR location LIKE '%query%'
    // Merge: union both result sets, deduplicate by internship_id, prefer higher score
    RETURN mergeAndDeduplicate(semanticResults, keywordResults)
```

AI ENGINE DATABASE TABLES

The `internship_embedding`, `student_embedding`, `notification_preference`, `internship_invitation`, and `internship_view` tables are defined in the DATABASE SCHEMA section above. They are not repeated here to avoid duplication.

AI API USAGE BUDGET (Free Tier)

Operation	API Used	When Called	Frequency	Daily Estimate
Skill extraction from resume	Gemini 2.5 Flash-Lite (generation)	Student uploads/re-uploads resume	Rare — a few per day	~5-20 requests
Skill extraction from internship description	Gemini 2.5 Flash-Lite (generation)	Employer clicks "Analyze Description" or "Re-analyze Description"	Rare — a few per day	~5-15 requests
Internship embedding	Gemini text-embedding-004	Admin approves internship or employer edits one	Rare — a few per day	~5-10 requests
Student embedding	Gemini text-embedding-004	Student updates profile or uploads resume	Rare — a few per day	~5-20 requests
Search query embedding	Gemini text-embedding-004	Every time a student searches	Moderate	~50-100 requests
Total daily estimate				**~70-165 requests**
Free tier limit				**Flash-Lite: 1000 RPD / Embeddings: 14400 RPD**

The system operates well within free tier limits, even with 50+ active users.

When Matching Runs:

- **Student recommendations**: When student visits /recommendations page → load cached student embedding + all internship embeddings → compute skill + semantic scores → sort descending → return top 20
- **Employer candidate ranking**: When employer clicks "View Candidates" for an internship → load internship embedding + all student embeddings → compute scores → sort descending

- **On application submit**: Calculate and store the final match_score in the application record
- **Embedding refresh**: Triggered on profile update, resume upload/replace, or internship edit — NOT on every page load. Resume upload is the most critical trigger because the entire student embedding and skill profile depends on it.

Match Score Detail Breakdown (for "Show Details" view):

When a student clicks to see details of their match score, return:

```
{
  "overall_score": 87.5,
  "skill_match_score": 90.0,
  "semantic_similarity_score": 82.3,
  "profile_bonus_score": 85.0,
  "matched_mandatory_skills": ["JavaScript", "React", "Node.js"],
  "missing_mandatory_skills": ["TypeScript"],
  "matched_optional_skills": ["Git", "Tailwind CSS"],
  "missing_optional_skills": ["Docker"],
  "proficiency_alignment": {
    "JavaScript": {"student": "advanced", "required": "intermediate", "status": "exceeds"},
    "React": {"student": "intermediate", "required": "intermediate", "status": "meets"},
    "Node.js": {"student": "beginner", "required": "intermediate", "status": "below"}
  },
  "gpa_bonus": 20,
  "major_relevance": true,
  "semantic_insight": "Your profile emphasizes web development and frontend technologies, which
aligns well with this full-stack internship role.",
  "improvement_tip": "Adding TypeScript to your skills would increase your match to approximately
93%."
}
```

`semantic_insight` **generation (rule-based, not LLM):** Constructed from a template using the student's top skill categories and the internship's industry/title. Pattern: "Your profile emphasizes {top 2-3 student skill categories}, which {aligns well with / partially aligns with / does not closely align with} this {internship title} role." The alignment assessment is based on the semantic similarity score: ≥70 = "aligns well", ≥40 = "partially aligns", <40 = "does not closely align". No LLM call — this is string interpolation.

`improvement_tip` **generation (rule-based, not LLM):** Identifies the highest-impact missing mandatory skill (the first one in `missing_mandatory_skills`). Estimates the score increase by re-running the skill match formula with that skill added at the required level. Pattern: "Adding {skill_name} to your skills would increase your match to approximately {estimated_score}%." If there are no missing mandatory skills, use: "You match all mandatory skills! Consider adding {first missing optional skill} to strengthen your profile." If no skills are missing at all, use: "Great match — you have all the skills this internship requires."

COMPLETE PAGE-BY-PAGE SPECIFICATION

Universal Visibility Rules

These rules apply to ALL pages, components, and API endpoints that show internships or users to students or the public:

- **Internships shown to students/public** must satisfy: `internship.status = 'active' AND`

`employer.users.is_active = TRUE AND (internship.deadline IS NULL OR internship.deadline >= CURDATE())`. This applies to: search results, featured internships, recommendations, homepage cards, internship detail pages, and any "top matches" widgets.
- **Students shown to employers** (in candidate recommendations or browsing) must satisfy: `student.users.is_active = TRUE`. Deactivated students are hidden from proactive candidate discovery. However, already-submitted applications from deactivated students remain visible in historical applicant lists.
- **Messaging with deactivated users:** Existing conversation history remains visible, but new messages cannot be sent to/from deactivated users. The chat input shows "This user's account is currently unavailable" instead of the message input field.
- **Admin dashboards** may optionally show all records (including deactivated/suspended) with a filter toggle.

Public Pages (No Authentication Required)

1. Landing Page (`/`)

- Hero section with headline: "Land Your Dream Internship" and subtext about AI-powered matching
- Two CTA buttons: "Get Started Free" → /register, "See How It Works" → smooth scroll to features section
- Stats bar: dynamically pulled from database — total active students, total active companies, active internships (from active employers only, deadline not passed), average match score. Deactivated/suspended accounts are excluded from public-facing counts.
- "How It Works" section: 3-step visual (1. Create Profile & Upload Resume → 2. AI Analyzes Your Skills → 3. Get Matched with Internships)
- Featured internships section: show 4-6 most recent active internships as preview cards
- Testimonials section (can be static/placeholder)
- Footer with links, copyright, social media icons

2. Registration Page (`/register`)

- Toggle between "Student" and "Employer" registration (tab or radio button at top)
- **Student Registration Form:**
- Full Name (required)
- Email (required, must be valid format)
- Password (required, min 8 chars, must contain 1 uppercase, 1 number)
- Confirm Password (must match)
- University (required, text input)
- Major (required, text input)
- Graduation Year (required, dropdown: 2000 to current year + 5. This wide range supports alumni and current students alike. Backend CHECK constraint enforces `graduation_year BETWEEN 2000 AND 2100`.)
- Terms & Conditions checkbox (required)
- **Employer Registration Form:**
- Full Name (required) — contact person
- Email (required)
- Password + Confirm Password
- Company Name (required)
- Industry (required, dropdown: Technology, Finance, Healthcare, Education, Marketing, Engineering, Other)

- Company Size (required, dropdown: 1-50, 51-200, 201-500, 500+)
- Terms & Conditions checkbox
- After registration: redirect to login page with success toast "Account created! Please log in."
- Link to login page: "Already have an account? Sign In"

3. Login Page (`/login`)

- Email field
- Password field (with show/hide toggle)
- "Remember Me" checkbox (stores JWT longer: 30 days vs. 24 hours)
- "Forgot Password?" link → /forgot-password
- Login button — on success, redirect based on role:
- Student → /student/dashboard
- Employer → /employer/dashboard
- Admin → /admin/dashboard
- Link to register: "Don't have an account? Sign up for free"
- Error handling: show specific messages ("Invalid email or password", "Account deactivated — contact admin")

4. Forgot Password (`/forgot-password`)

- Email input field
- "Send Reset Link" button
- Backend: generates a cryptographically random token, stores in password_reset_token table (expires in 1 hour), sends email via Nodemailer with reset link
- Reset link format: `/reset-password?token=abc123`
- Success message: "If an account with this email exists, a reset link has been sent." (don't reveal whether email exists)

5. Reset Password (`/reset-password?token=...`)

- Validates token exists, is not expired, and is not used
- New Password + Confirm New Password fields
- On success: marks token as used, updates user password (bcrypt hashed), redirects to login with "Password reset successful"

Student Pages (Protected — role: student)

6. Student Dashboard (`/student/dashboard`)

- **Welcome banner**: "Welcome back, {name}!" with notification count
- **Stats row**: Applications (count from application table), Saved Internships (count from bookmark table), Skills Count (number of skills in profile)
- **Profile Strength indicator**: percentage bar calculated from how many profile fields are filled (name, major, university, GPA, bio, resume uploaded, skills added, LinkedIn URL)
- **Recommended For You section**: Top 5 internships by match score. Each card shows:

- Company logo + company name
- Internship title
- Location + work type (remote/hybrid/on-site)
- Salary range (if provided)
- Match percentage badge (visually differentiated by tier: high ≥80%, medium ≥50%, low <50%)
- "View Details" button
- Bookmark icon (heart/star)
- **Recent Applications section**: Last 3-5 applications with status badge (color-coded by status), company name, role, date applied
- **Quick Actions**: "Browse All Internships", "Track Applications", "Update Profile"

7. Student Profile (`/student/profile`)

- **Profile header**: Profile picture (upload/change), full name, university, major, GPA
- **Editable sections** (each with save button):
- Personal Info: name, email (read-only), phone, LinkedIn URL, GitHub URL, Instagram URL, bio
- Academic Info: university, university start date, major, GPA, graduation year
  - **Graduation status** is auto-displayed based on graduation_year: shows "Graduated" if graduation_year < current year, "Expected graduation: {year}" otherwise. Not editable directly — derived from graduation_year.
- Skills section:
  - Display all current skills as tags with proficiency level badge and source indicator (manual/extracted)
  - "Add Skill" button → opens searchable dropdown from skill table + proficiency level selector
  - Each skill tag has an X to remove and a click to change proficiency level
  - If resume has been uploaded, show "Re-extract from Resume" button
- Resume section:
  - Show current resume filename, upload date, or "No resume uploaded" state
  - "Upload Resume" button if no resume exists / "Replace Resume" button if one exists (accepts .pdf, .docx only, max 5MB)
  - **Upload flow:** Replacing an existing resume triggers the full replacement transaction: delete old resume → insert new → re-extract skills → re-generate embedding. The student is shown extracted skills from the new resume for review ("We found these skills in your resume — confirm or edit:")
  - Student can accept all, deselect some, or add more before saving
  - "Delete Resume" button (with confirmation) — removes the resume entirely, clears primary_resume_id, nulls resume_id on existing applications (snapshots preserved)
  - **Resume is required for applying:** The "Apply" button on internship pages is disabled with a tooltip "Upload a resume to apply" if no resume exists.
- **Account Settings** link → /student/settings

8. Internship Listings (`/student/internships`)

- **NLP-Powered Search bar**: Free text search that uses **semantic search** (see Subsystem 3 in AI Engine spec). Students can type natural language queries like "I want to build mobile apps" or "data analysis work in Amman" — the system understands intent, not just keywords.

- Powered by Gemini text embeddings + cosine similarity under the hood
- Falls back to hybrid (semantic + keyword SQL) for short/exact queries
- Shows a "Powered by AI" badge next to the search bar
- Autocomplete suggestions from recent searches and skill names
- **Filter sidebar/panel** (applied ON TOP of semantic ranking):
- Location (text input with autocomplete or dropdown)
- Work Type: checkboxes for Remote, Hybrid, On-Site
- Industry: dropdown (matches employer industries)
- Duration: range slider or dropdown (1-3 months, 3-6 months, 6-12 months, 12-24 months). Covers the full schema range of 1-24 months.
- Salary: "Paid only" checkbox
- Sort by: dropdown (Relevance — default when searching, Match Score, Newest, Deadline Soon)
- **Results grid/list**: Internship cards showing:
- Company logo + name
- Internship title
- Location + work type
- Duration
- Salary range (or "Not specified" if both salary_min and salary_max are NULL — do NOT display as "Unpaid" since null salary means unspecified, not necessarily unpaid)
- Posted date
- Deadline (if set) with urgency indicator if <7 days away
- Match score percentage badge (from AI matching engine)
- Relevance score indicator (when searching — "Highly Relevant", "Relevant", "Somewhat Relevant")
- Bookmark toggle
- "Apply" or "View Details" button
- **Pagination**: 10 internships per page or infinite scroll
- **Empty state**: "No internships match your search. Try describing what you're looking for differently — our AI understands natural language!"

9. Internship Details (`/internship/:id`) — Public

- **Publicly accessible** — no login required to view internship information. This allows sharing internship links externally. Action buttons (Apply, Bookmark, Message) are visible but prompt login if user is not authenticated.
- **Full internship details** (shown to everyone):
- Title, company name + logo, location, work type, duration, salary range, posted date, deadline
- Full description (plain text, rendered with `white-space: pre-wrap` to preserve line breaks and paragraphs)
- Company info: description, industry, size, location, website link, social media links (LinkedIn, Twitter/X, Facebook, Instagram — shown as icons, only if provided)
- **Required Skills section** (adapts based on viewer):
- **If logged-in student:** skill tags are visually differentiated against the student's profile:
  - Student has this skill at or above required level (positive indicator)
  - Student has this skill but below required level (partial indicator)
  - Student doesn't have this skill (missing indicator)

  - Each tag shows "(Mandatory)" or "(Nice to have)"
- **If anonymous, employer, or admin:** skill tags shown in a neutral style (no comparison), each labeled "(Mandatory)" or "(Nice to have)"
- **Match Score section** (student-only):
- **If logged-in student:** Large percentage display (e.g., "87% Match") + "Show Details" expandable panel showing the full breakdown (skill score, semantic score, matched skills, missing skills, improvement tip)
- **If anonymous:** Show a CTA banner: "Log in as a student to see your personalized AI match score"
- **If employer or admin:** Section is hidden entirely
- **Action buttons** (adapt based on viewer):
- **If logged-in student:** "Apply Now" → opens application modal (if not already applied), "Bookmark" toggle, "Message Employer" → opens/creates conversation
- **If anonymous:** Same buttons shown but clicking any of them redirects to `/login?redirect=/internship/:id`
- **If employer (own internship):** "Edit Internship" button instead
- **If employer (other's internship) or admin:** No action buttons
- **Application modal**:
- Shows the student's current resume filename (read-only — no selection, single-resume model)
- Optional cover letter text area
- "Submit Application" button
- **Pre-conditions checked before modal opens:** (1) Student has a resume uploaded — if not, show "You need to upload a resume before applying" with a link to profile page. (2) Internship status is 'active'. (3) Internship deadline has not passed. (4) Student has not already applied — if they have, show "You applied on {date} — Status: {status}" instead of the modal.
- If already applied: show "You applied on {date} — Status: {status}" instead
- **Log view event:** On page load, insert into `internship_view` table ONLY if the viewer is a student (not the internship's own employer or an admin reviewing it). Deduplicated: one entry per student per internship per hour, tracked via viewer_user_id.

10. Recommendations Page (`/student/recommendations`)

- Full-page dedicated to AI recommendations
- Shows ALL internships sorted by match score (descending), not just top 5
- Same card layout as Internship Listings but sorted by score
- Filter: minimum match score slider (e.g., "Show only ≥50% match")
- "Why this match?" expandable on each card → shows skill breakdown

11. My Applications (`/student/applications`)

- Table/list view of all applications:
- Internship title + company
- Applied date
- Match score
- Status badge (visually distinct per status):
  - Pending, Under Review, Interview Scheduled, Accepted, Rejected, Withdrawn — each with a unique visual treatment

- Actions: "View Details", "Withdraw" (only if status is pending, under_review, or interview_scheduled per the state machine)
- Filter by status dropdown
- Sort by date applied
- Click row → navigates to internship details page. If the internship is no longer active (closed, rejected, or employer suspended), the page still loads for this student (as an authorized related user) but shows a "This internship is no longer active" banner at the top. Apply button is hidden; other details remain visible for reference.

12. Saved Internships (`/student/saved`)

- Grid of bookmarked internships (same card layout)
- Remove bookmark button on each
- Empty state: "You haven't saved any internships yet. Browse internships to find opportunities."

13. Messages (`/student/messages`)

- Left sidebar: list of conversations (employer name, company, last message preview, timestamp, unread indicator)
- Right panel: active conversation chat view
- Message bubbles (student's on right, employer's on left)
- Timestamp on each message
- Text input at bottom with send button
- Real-time updates via Socket.IO (new messages appear instantly)
- "Type a message..." placeholder
- If no conversations: "No messages yet. Start a conversation from an internship page."
- When a new message is received: push notification (toast) + increment bell icon badge

14. Student Settings (`/student/settings`)

- **Account section**: Change password (current password + new password + confirm)
- **Notification preferences**: Toggle email notifications on/off for:
- Application status changes (`email_application_status`)
- New internship recommendations (`email_recommendations`)
- New messages (`email_new_message`)
- Invitation to apply (`email_invitation`)
- **Theme**: Light / Dark mode toggle (persisted in localStorage)
- **Danger zone**: "Delete My Account" button → confirmation modal ("This will permanently delete your account and all user-owned records including your profile, resume, applications, messages, and bookmarks. Some anonymized audit or analytics records may remain. This cannot be undone.") → calls `DELETE /api/student/account`

15. Student Notifications (`/student/notifications`)

- Full-page list of all notifications for the student, paginated (20 per page)
- Each notification row: icon (by type), title, message, timestamp, read/unread visual indicator

- Click a notification → marks as read + navigates to the relevant entity (internship, application, etc.). If the target no longer exists, show inline "This item is no longer available."
- "Mark all as read" button at top
- Filter: "Unread only" toggle
- Sorted by `created_at DESC`
- Uses the same `GET /api/student/notifications` endpoint already defined in the API table

---

Employer Pages (Protected — role: employer)

16. Employer Dashboard (`/employer/dashboard`)

- **Welcome banner**: "Welcome back, {company_name}!" with notification count
- **Stats row**: Active Posts (count), Total Applicants (count across all internships), Accepted (count of applications with status='accepted' across all internships), Internship Views (total COUNT from internship_view for all their internships — this is how many times students viewed their listings, not company profile views)
- **AI Top Matches section**: Top 3-5 recommended candidates across all their active internships (student name, university, major, GPA, matched internship, score)
- **Active Internship Posts section**: Each internship card showing:
- Title, status badge (pending/active/closed)
- Stats: applicants count, viewed count (from `internship_view`), accepted count (applications with status `accepted`)
- "View Applicants" button
- "Edit" button
- **Recent Activity feed**: Last 5 events (new application received, candidate accepted, internship approved by admin, etc.)
- **Quick Actions**: "Post New Internship", "Browse Candidates", "View All Internships"

17. Post Internship (`/employer/internship/new`)

- Form with fields:
- Title (required)
- Description (required, text area — plain text only, supports paragraphs via line breaks. Rendered with `white-space: pre-wrap` on the frontend. No HTML, no markdown.)
- Location (required)
- Work Type: radio buttons (Remote, Hybrid, On-Site)
- Duration in months (required)
- Salary Range: min and max (optional, number inputs)
- Application Deadline (optional, date picker, must be in future)
- Required Skills section:
  - **"Analyze Description" button** — sends the description text to the AI skill extraction endpoint (Subsystem 1B). Enabled only when description has ≥50 characters. On click: shows a loading state, then populates the skills list below with AI-suggested skills (each marked with an "AI suggested" badge).

If skills were already manually added, extracted skills merge in (new ones appended, duplicates skipped — see merge logic in Subsystem 1B). Employer reviews and edits before saving.
  - Searchable dropdown pulling from `skill` table (manual skill picker — works alongside AI extraction)
  - "Add Skill" button → adds to list below
  - For each skill in the list: set required level (Beginner/Intermediate/Advanced) and toggle Mandatory/Nice-to-have. AI-suggested skills come pre-filled with the AI's estimates but are fully editable.
  - "Add Custom Skill" button → text input to add a new skill not in the database (inserts into skill table then adds to this internship)
  - Remove button (X) on each skill to remove it from the list
  - Minimum 1 skill required to post
- "Post Internship" button → saves with status 'pending_approval'
- Success message: "Your internship has been submitted for admin review. You'll be notified once approved."

18. Edit Internship (`/employer/internship/:id/edit`)

- Same form as Post Internship, pre-filled with existing data
- Can edit if status is `pending_approval`, `active`, or `rejected`
- **"Re-analyze Description" button** — appears only when the description text has been modified from the saved version. Works the same as "Analyze Description" on the post form: sends the updated description to the AI, merges extracted skills with existing required skills. Employer reviews before saving.
- If `active` and edited: stays active (no re-approval needed for minor edits)
- If `rejected` and edited: show the admin's rejection note at the top of the form so the employer knows what to fix. After editing, a **"Resubmit for Review"** button appears which sets status back to `pending_approval`. This triggers the rejection → edit → re-submit → re-approval cycle described in the AI embedding section.
- Cannot edit if status is `closed`. Show a **"Reopen Internship"** button instead → calls `PUT /internships/:id/reopen` which sets status back to `active` (validates deadline hasn't passed). Once reopened, the form becomes editable again.
- "Close Internship" button → sets status to 'closed'
- "Delete Internship" button → confirmation modal ("This will permanently delete this internship along with its applications, required skills, views, and invitations. Conversations linked to this internship will remain but lose their internship context. This cannot be undone.") → deletes internship (cascading to applications, required skills, views, invitations; conversations get internship_id set to NULL)

19. Manage Internships (`/employer/internships`)

- Table of all internships posted by this employer:
- Title, status, posted date, deadline, applicant count, views
- Actions: View, Edit, Close, Delete, Resubmit (shown only for rejected internships)
- Filter by status dropdown
- Sort by date

20. View Applicants (`/employer/internship/:id/applicants`)

- List/table of all students who applied to this specific internship:
- Student name, university, major, GPA
- **Graduation status badge:** "Graduated ({year})" or "Expected {year}" — visually distinct so employers can see at a glance

- Match score percentage badge
- Application date
- Current status badge
- Actions dropdown:
  - "View Profile" → expandable panel or modal showing student's full profile + skills + resume download link
  - "Change Status" → dropdown showing ONLY valid next statuses per the state machine (e.g., if current status is `pending`, show Under Review and Rejected only). Each option has an optional note text field.
  - "Download Resume" → direct download of the student's submitted resume file (uses `submitted_resume_path` snapshot, not the student's current resume — the student may have replaced their resume since applying)
  - "Message Student" → opens/creates conversation
- On status change: creates notification for student + sends email if enabled
- Sort by: Match Score (default), Applied Date, Status
- Filter by status

21. AI Candidate Recommendations (`/employer/candidates`)

- Select an internship from dropdown (only `active` internships shown)
- Shows all active students in the system ranked by match score for that internship (not just applicants — proactive talent discovery)
- Student cards: name, university, major, GPA, skills (top 5), match score, **graduation status badge**
- "Invite to Apply" button → sends a notification to the student about this internship. Disabled if invitation already sent (shows "Invited" badge instead).
- Minimum score filter slider

22. Employer Company Profile (`/employer/profile`)

- View/edit company information:
- Company Name, Industry, Company Size, Description (plain text area)
- Company Logo upload (max 2MB, jpg/png)
- Location (company headquarters)
- Social & Web links (all optional): Website URL, LinkedIn URL, Twitter/X URL, Facebook URL, Instagram URL
- Contact person (full_name, email)
- Preview: how the company appears to students on internship cards and detail pages

23. Employer Analytics (`/employer/analytics`)

- Per-internship statistics:
- Bar chart: applicants per internship
- Pie chart: application status distribution
- Line chart: views over time by week (query: `SELECT YEARWEEK(viewed_at), COUNT(*) FROM internship_view WHERE internship_id = ? GROUP BY YEARWEEK(viewed_at)`)
- Overall statistics:
- Total applicants across all posts
- Average match score of applicants

- Conversion rate (applicants → accepted)
- Most in-demand skills (from applicant pool)
- Built using Recharts

24. Employer Messages (`/employer/messages`)

- Same layout as student messages but from employer perspective
- Lists all conversations with students

25. Employer Settings (`/employer/settings`)

- **Account section**: Change password (current password + new password + confirm)
- **Notification preferences**: Toggle email notifications on/off for:
- New applications received (`email_new_application`)
- Internship approved/rejected by admin (`email_internship_approved`)
- New messages (`email_new_message`)
- **Theme**: Light / Dark mode toggle (persisted in localStorage)
- **Danger zone**: "Delete My Account" button → confirmation modal → calls `DELETE /api/employer/account`

26. Employer Notifications (`/employer/notifications`)

- Full-page list of all notifications for the employer, paginated (20 per page)
- Each notification row: icon (by type), title, message, timestamp, read/unread visual indicator
- Click a notification → marks as read + navigates to the relevant entity (internship, application, etc.). If the target no longer exists, show inline "This item is no longer available."
- "Mark all as read" button at top
- Filter: "Unread only" toggle
- Sorted by `created_at DESC`
- Uses the same `GET /api/employer/notifications` endpoint already defined in the API table

---

Admin Pages (Protected — role: admin)

27. Admin Dashboard (`/admin/dashboard`)

- **System Overview stats cards**:
- Total Users (students + employers + admins)
- Active Internships
- Pending Reviews (internships awaiting approval)
- Total Applications
- System Health indicator (always "Operational" for this version)
- **Pending Reviews section**: List of internships with status 'pending_approval'
- For each: title, company, posted date, quick "Approve" / "Reject" buttons
- "Review" button → opens detail modal with full internship info + "Approve" / "Reject with note" actions
- On approve: sets status to 'active', notifies employer via email + in-app notification
- On reject: sets status to 'rejected', saves admin note, notifies employer

- **Recent Activity feed**: Last 10 system log entries (user registrations, logins, internship posts, applications)
- **Quick Actions**: "Manage Users", "Review Internships", "View Reports", "View Logs"

28. User Management (`/admin/users`)

- Table of ALL users:
- Name, email, role, is_active status, date created
- Search bar (search by name or email)
- Filter by role (Student, Employer, Admin, All)
- Filter by status (Active, Deactivated, All)
- Actions per user:
- "View Details" → modal with full profile info
- "Edit" → edit user info (admin can change any field)
- "Deactivate" / "Activate" toggle → sets is_active (prevents login without deleting data)
- "Delete" → permanent delete with confirmation modal ("This will permanently delete this user and all their owned data. Some anonymized audit or analytics records may remain.") → deletes user (cascading)
- Pagination: 20 users per page

29. Internship Management (`/admin/internships`)

- Table of ALL internships across all employers:
- Title, company, status, posted date, applicant count
- Filter by status
- Search by title or company
- Actions: View, Approve, Reject, Delete
- Same approve/reject flow as dashboard

30. System Logs (`/admin/logs`)

- Searchable, filterable table of system_log entries:
- Timestamp, user (name + role), action, IP address, details
- Filter by action type dropdown
- Filter by date range (date pickers)
- Search by user name
- Pagination: 50 per page
- "Export CSV" button → generates and downloads CSV file of filtered logs

31. Reports (`/admin/reports`)

- **Generate Report** form:
- Report Type dropdown:
  - User Activity Report
  - Internship Applications Report
  - Student Match Ranking Report
  - Employer Performance Report
  - System Audit Report

- Date range (start date, end date)
- Additional filters based on type (e.g., specific employer, specific internship)
- "Generate" button
- Report output:
- Displays data in a table on screen
- "Download as PDF" and "Download as CSV" buttons — these generate the export on demand from the displayed data, no files are stored on disk
- Reports are **generated on demand** — the DB is queried fresh each time, no output files are stored on disk. Each generation inserts a row into the `report` table as an audit log (who requested, what type, what filters, when). The "Download as PDF" and "Download as CSV" buttons generate the export on demand from the displayed data.

32. Admin Settings (`/admin/settings`)

- **Account section**: Change password (current password + new password + confirm)
- **Theme**: Light / Dark mode toggle (persisted in localStorage)
- **No notification preferences** — admin notifications are always delivered, there are no opt-out toggles for admins
- **No "Delete My Account"** — admin accounts are managed by other admins, not self-deleted

33. Admin Notifications (`/admin/notifications`)

- Full-page list of all notifications for the admin, paginated (20 per page)
- Each notification row: icon (by type), title, message, timestamp, read/unread visual indicator
- Click a notification → marks as read + navigates to the relevant entity (internship, user, etc.). If the target no longer exists, show inline "This item is no longer available."
- "Mark all as read" button at top
- Filter: "Unread only" toggle
- Sorted by `created_at DESC`
- Uses the same `GET /api/admin/notifications` endpoint already defined in the API table

---

Shared Components

Navigation Bar (present on all authenticated pages)

- Logo "InternMatch" → links to dashboard
- Role badge next to logo (Student/Employer/Admin)
- Search bar (global internship search — **visible to students and employers**, hidden for admins since admins have their own internship management page with separate filters). **Intentional design:** employers can see other companies' active internship postings. This is standard practice on job platforms — it helps employers benchmark their offerings against competitors, understand market positioning, and see what skills other companies are requesting. Employers see listings without match scores (since they have no student profile to match against).
- **Bell icon** with unread notification count badge (all roles):
- Click → dropdown showing recent notifications

- Each notification: icon, title, message, timestamp, read/unread indicator
- Click a notification → marks as read + navigates to relevant page. **Graceful handling for deleted targets:** if the referenced entity (internship, application, etc.) no longer exists (because it was permanently deleted), show an inline message "This item is no longer available" instead of a broken page or error. The notification itself remains visible in the list.
- "Mark all as read" link
- "View All Notifications" link → navigates to the role's notifications page (`/student/notifications` , `/employer/notifications` , or `/admin/notifications`)
- **Messages icon** with unread count badge → links to messages page. **Visible to students and employers only.** Admins do not have a messaging feature — the icon is hidden for admin role.
- **Profile avatar** dropdown:
- "My Profile" → links to profile page. **Visible to students and employers only.** For admins, this item is hidden (admins do not have a dedicated profile page — admin account info is managed via the admin settings page).
- "Settings"
- Dark/Light mode toggle
- "Logout"
- Fully responsive: collapses to hamburger menu on mobile

Dark/Light Mode

- Toggle in navbar and in settings
- Persisted in localStorage key "theme"
- Default: follow system preference on first visit
- Must define a consistent light and dark palette (exact colors to be determined during frontend design phase)

404 Page

- Friendly "Page Not Found" message with link back to dashboard

Loading States

- Skeleton loading placeholders on all pages while data loads
- Button loading spinners during form submissions
- Full-page loading spinner for initial auth check

Error States

- Toast notifications for all errors (network, validation, server)
- Form-level inline error messages below each field
- "Something went wrong" fallback component for unexpected errors

---

API ENDPOINTS — COMPLETE LIST

Auth Routes (`/api/auth`)

Method	Endpoint	Body/Params	Description	Auth Required
POST	/register	{ fullName, email, password, role, ...roleFields }	Register new user	No
POST	/login	{ email, password, rememberMe }	Returns JWT token + user info	No
POST	/forgot-password	{ email }	Sends password reset email	No
POST	/reset-password	{ token, newPassword }	Resets password	No
GET	/me	—	Returns current user info from the **database** (not just JWT claims). Queries the user's full profile including role, is_active, full_name, email, profile_picture, and role-specific fields. This ensures the frontend always has the latest data even if the JWT was issued before a profile change. Requires valid JWT.	Yes
POST	/logout	—	Client-side only: frontend removes the JWT from localStorage and clears auth context state. No server-side action needed for stateless JWT. To force-logout all sessions, use the change-password or admin toggle-active flows which increment `token_version`. Logs the logout action to system_log.	Yes

Student Routes (`/api/student`)

Method	Endpoint	Description	Auth
GET	/profile	Get current student's full profile (includes graduation_status derived from graduation_year)	Student
PUT	/profile	Update student profile fields. **Triggers:** if skills, bio, major, or university change, regenerate student embedding. `graduation_status` is never stored — it is computed on read from `graduation_year`.	Student
POST	/profile/picture	Upload or replace profile picture (max 2MB, jpg/png only). Old picture file deleted from disk as best-effort.	Student

Method	Endpoint	Description	Auth
POST	/resume/upload	Upload or replace resume file (multipart). **Two-phase flow:** Phase 1 (this endpoint): saves the file to disk, extracts text via pdf-parse/mammoth, runs LLM skill extraction, and returns the extracted skills for review — but does NOT yet commit the resume replacement to the DB. The old resume (if any) remains active. The new file is saved to a staging path. Phase 2 (`POST /resume/confirm`): the student confirms the extracted skills, the backend runs the replacement transaction (delete old → insert new → set primary → save confirmed skills → regenerate embedding), and the resume becomes active. **If the student cancels or navigates away without confirming:** the staged file is orphaned and cleaned up by a periodic cleanup job (or on next upload attempt). Max 5MB, .pdf/.docx only.	Student
POST	/resume/confirm	Confirm a staged resume upload. **Request body:** `{ extractedSkills: [...] }` — the skills the student accepted during review (may be a subset of what the LLM extracted, may include edits to proficiency levels). Runs the full replacement transaction: clears old primary_resume_id → nulls application.resume_id on old applications → deletes old resume record → inserts new resume from staging → sets as primary → saves confirmed skills to has_skill (inserting new skills into skill table only at this point) → regenerates student embedding → COMMIT. Old resume file is deleted only if no applications reference it (reference-counted).	Student
GET	/resume	Get current resume info (filename, upload date, file_type) or 404 if none	Student
DELETE	/resume	Delete the student's resume. **DB changes run inside a transaction:** (1) clear `student.primary_resume_id` , (2) null `application.resume_id` on any applications referencing this resume (submitted snapshots are preserved), (3) delete the resume DB record, (4) COMMIT. **After DB commit:** check if any `application.submitted_resume_path` still references the old file — if yes, keep the file (it's needed for submitted application downloads). If no references remain, delete the physical file. Also clears extracted skills with source='extracted' and regenerates student embedding (now without resume data).	Student
GET	/skills	Get student's current skills	Student
POST	/skills	Add skill(s) to profile	Student
PUT	/skills/:skillId	Update skill proficiency level	Student
DELETE	/skills/:skillId	Remove a skill	Student
POST	/skills/extract	Re-extract skills from current resume via LLM. Returns extracted skills for review. Requires a resume to be uploaded.	Student
GET	/recommendations	Get AI-matched internships sorted by score. **Filter:** applies the canonical public visibility filter (see DATABASE SCHEMA section).	Student
GET	/applications	List all student's applications (paginated, filterable by status, sortable by date)	Student
GET	/applications/:id	Get full application details including internship info, match score breakdown, and current status	Student
GET	/applications/:id/history	Get the full status history for an application from `application_status_history` table. Returns array of `{ oldStatus, newStatus, changedBy, note, createdAt }` sorted by createdAt ASC.	Student

Method	Endpoint	Description	Auth
POST	/applications	Apply to an internship. **Pre-validations:** (1) student must have a resume uploaded — return 400 "Upload a resume before applying" if not, (2) internship must have status='active' — return 400 if not, (3) internship deadline must not have passed — return 400 "Application deadline has passed" if `deadline < CURDATE()`, (4) student must not have already applied — return 409 "You have already applied to this internship" on duplicate key. **Also:** calculates and stores match_score, inserts initial row into `application_status_history` with old_status = NULL, new_status = 'pending', changed_by = student. Triggers notification + email to employer.	Student
PUT	/applications/:id/withdraw	Withdraw an application. Sets status = 'withdrawn', inserts row into `application_status_history` with changed_by = student	Student
GET	/bookmarks	List all bookmarked internships	Student
POST	/bookmarks/:internshipId	Bookmark an internship	Student
DELETE	/bookmarks/:internshipId	Remove bookmark	Student
GET	/invitations	List all invitations for this student. **Paginated:** query params `page` (default 1) and `limit` (default 20). Sorted by `created_at DESC`.	Student
PUT	/invitations/:id/view	Mark an invitation as viewed. Only allowed if current status is `pending`. Sets status to `viewed`.	Student
PUT	/invitations/:id/dismiss	Dismiss an invitation. Sets status = 'dismissed'. Student can still apply to the internship independently.	Student
GET	/notifications	List notifications for this student. **Paginated:** query params `page` (default 1) and `limit` (default 20, max 50). Returns `{ notifications, total, page, totalPages }`. Sorted by `created_at DESC`. Optional filter: `?unread=true` to show only unread.	Student
PUT	/notifications/:id/read	Mark notification as read	Student
PUT	/notifications/read-all	Mark all notifications as read	Student
PUT	/change-password	Change password (requires current password + new password). NOT the same as forgot/reset password — this is for authenticated users. **Also increments `users.token_version`** to invalidate all existing JWT sessions.	Student
GET	/notification-preferences	Get current notification preference toggles	Student
PUT	/notification-preferences	Update notification preference toggles	Student
DELETE	/account	Permanently delete own account. Requires password confirmation in request body. Deletes user row and all owned data cascades (profile, resume, applications, messages, bookmarks, skills, embeddings). **Also cleans up uploaded files** from storage: profile picture, resume file. Anonymized audit/analytics rows (system_log, internship_view) remain with user_id = NULL. File cleanup is best-effort — orphaned files are logged for scheduled cleanup if deletion fails.	Student

Employer Routes (`/api/employer`)

Method	Endpoint	Description	Auth
GET	/profile	Get employer company profile	Employer
PUT	/profile	Update company profile	Employer
POST	/profile/picture	Upload or replace profile picture for the employer contact person (max 2MB, jpg/png only). Old picture file deleted as best-effort.	Employer
POST	/profile/logo	Upload company logo (max 2MB, jpg/png only). Old logo file deleted as best-effort.	Employer
POST	/internships	Create new internship posting	Employer
POST	/internships/extract-skills	Extract required skills from internship description text using AI (Subsystem 1B). **Request body:** `{ description }` (minimum 50 characters). Returns extracted skills with `skill_name`, `category`, `required_level`, `is_mandatory`, and `source: 'extracted'` for employer review. Does NOT save anything — the employer reviews and submits the skills with the internship form. **Rate limited:** max 10 per hour per user. **Fallback:** if Gemini is unavailable, returns 503 with "AI analysis temporarily unavailable. Please add skills manually."	Employer
GET	/internships	List all internships by this employer	Employer
GET	/internships/:id	Get specific internship details	Employer
PUT	/internships/:id	Update internship. Allowed when status is `pending_approval`, `active`, or `rejected`. Returns 400 if `closed`. **If status is `active` and description/ title/skills change:** regenerates the internship embedding via Gemini Embedding API. This ensures semantic search and matching stay current without requiring re-approval for minor edits.	Employer
PUT	/internships/:id/resubmit	Resubmit a rejected internship for admin review. Only allowed when status is `rejected`. Sets status to `pending_approval` and clears `admin_review_note`.	Employer
PUT	/internships/:id/close	Close an internship (sets status = 'closed')	Employer
PUT	/internships/:id/reopen	Reopen a closed internship. Only allowed when status is `closed`. Sets status back to `active`. **Validations:** deadline must not have passed (if deadline is set and past, return 400 "Cannot reopen — the deadline has passed. Create a new posting instead."). **Triggers:** regenerates internship embedding if one existed previously.	Employer
DELETE	/internships/:id	Permanently delete an internship. Cascades to applications, required skills, views, invitations. Conversations linked to this internship remain with internship_id = NULL	Employer
GET	/internships/:id/applicants	List all applicants for an internship. **Returns for each applicant:** student name, email, university, major, GPA, graduation status (computed), profile picture URL, skills (top 5 with proficiency), match score, application date, current status, cover letter, submitted resume filename. This is the full payload needed for the "View Profile" modal and applicant list — no separate detail endpoint needed. **Paginated.** Sortable by match_score, applied_date, status. Filterable by status.	Employer

Method	Endpoint	Description	Auth
GET	/applications/:id/resume	Download the submitted resume file for a specific application. **Access control:** only the owning employer (whose internship the application belongs to) or an admin can download. Uses `submitted_resume_path` (the immutable snapshot from application time), NOT the student's current resume. Returns the file as a download stream with `Content-Disposition: attachment`. Returns 404 if the physical file no longer exists on disk (edge case — should not happen with reference-counted retention, but handle gracefully).	Employer, Admin
PUT	/applications/:id/status	Update application status + optional note. **State machine enforced:** only transitions defined in the application status state machine are allowed (see APPLICATION table section). Returns 400 for invalid transitions. **Also:** inserts row into `application_status_history` with old_status, new_status, changed_by_user_id, and note. Updates `application.status_updated_at`. Triggers notification + email to student.	Employer
GET	/internships/:id/candidates	AI-ranked candidates for an internship. **Validations:** internship must belong to the requesting employer AND have `status = 'active'`. Returns 403 if not owned, 400 if internship is closed/rejected/pending. **Filter:** only students with `users.is_active = TRUE`. Deactivated students are excluded from proactive candidate discovery.	Employer
POST	/internships/:internshipId/invite/:studentId	Invite a student to apply for a specific internship. **Validations:** internship must belong to the requesting employer, must be `active`, student must be active, no existing invitation for this student+internship pair. Creates invitation with status `pending`, sends notification + email to student.	Employer
GET	/analytics	Get analytics data for all internships	Employer
GET	/notifications	List notifications for this employer. **Paginated:** same params as student notifications.	Employer
PUT	/notifications/:id/read	Mark notification as read	Employer
PUT	/notifications/read-all	Mark all notifications as read	Employer
PUT	/change-password	Change password (requires current password + new password). **Also increments** `users.token_version` to invalidate all existing JWT sessions.	Employer
GET	/notification-preferences	Get current notification preference toggles	Employer
PUT	/notification-preferences	Update notification preference toggles	Employer
DELETE	/account	Permanently delete own account. Requires password confirmation. Deletes user row and all owned data cascades (profile, internships, applications received, messages, logo). **Also cleans up uploaded files** from storage: profile picture, company logo. Anonymized audit/analytics rows remain with user_id = NULL. File cleanup is best-effort.	Employer

Internship Routes (`/api/internships`) — Public/Mixed

Method	Endpoint	Description	Auth
GET	/	List active internships with NLP semantic search + filters + pagination. Query param `q` triggers semantic search via Gemini embeddings; without `q` , returns all with traditional filters. **Filters applied server-side:** uses the canonical public visibility filter (status='active', employer active, deadline not passed). **Auth: optional.** Uses optional auth middleware — if a valid JWT is present, the user is identified; if not, the request proceeds as anonymous. **Role-based response:** if requester is a logged-in student, each result includes a personalized `match_score` badge. If requester is an employer, results are returned without match scores. If anonymous, no match scores.	Optional
GET	/:id	Get internship details. **Auth: optional** (same optional auth middleware). **Access rules:** (1) If internship passes the canonical public visibility filter (status='active', employer active, deadline not passed) → accessible to everyone (public). (2) If internship fails the visibility filter (closed, rejected, pending_approval, employer suspended, or past deadline) → returns 404 to anonymous/unrelated users, BUT still returns data to **authorized related users**: the student who applied to it, the owning employer, or an admin. This requires the optional auth check to identify the requester. Response includes a `status` field so the UI can show a "This internship is no longer active" banner. **Conditional personalization:** if requester is a logged-in student, response includes `match_score` , `skill_comparison` , and `match_breakdown` . Otherwise omitted. **View logging:** inserts into `internship_view` ONLY when viewer is a logged-in student viewing an internship that passes the canonical visibility filter.	Optional
GET	/featured	Get featured/recent internships for landing page. **Filter:** applies the canonical public visibility filter (see DATABASE SCHEMA section).	No
GET	/stats	Get platform stats for landing page. **Counting rules:** total students = `users WHERE role='student' AND is_active=TRUE` ; total companies = `users WHERE role='employer' AND is_active=TRUE` ; active internships = `internship WHERE status='active' AND employer is_active=TRUE AND (deadline IS NULL OR deadline >= CURDATE())` ; average match score = `AVG(match_score) FROM application WHERE applied_date >= DATE_SUB(CURDATE(), INTERVAL 90 DAY) AND status NOT IN ('withdrawn') AND match_score IS NOT NULL` — uses last 90 days, excludes withdrawn applications, requires at least 10 applications in the window to display (otherwise show nothing or a placeholder).	No

Messaging Routes (`/api/messages`)

Method	Endpoint	Description	Auth
GET	/conversations	List all conversations for current user. **Access:** returns only conversations where `student_user_id` or `employer_user_id` matches the authenticated user.	Yes
POST	/conversations	Create new conversation. **Permissions:** students can initiate conversations with any active employer; employers can initiate conversations with any active student. Both parties must have `is_active = TRUE` . **Deactivated user check:** if the other party is deactivated, return 400 "This user's account is currently unavailable." **Uniqueness:** enforced by `UNIQUE(student_user_id, employer_user_id, context_type, context_key)` — if conversation already exists, return the existing one instead of creating a duplicate.	Yes
GET	/conversations/:id/messages	Get messages in a conversation. **Access control:** the authenticated user MUST be a participant in this conversation (student_user_id or employer_user_id matches). Return 403 if not.	Yes

Method	Endpoint	Description	Auth
POST	/conversations/:id/messages	Send a message. **Access control:** same as above — user must be a participant. Additionally, if the other party's account is deactivated, return 400 "Cannot send messages — this user's account is currently unavailable." Messages from deactivated users are blocked at the API level.	Yes
PUT	/conversations/:id/read	Mark all messages in conversation as read. **Access control:** user must be a participant.	Yes

Skill Routes (`/api/skills`)

Method	Endpoint	Description	Auth
GET	/	List all skills (with optional search query matching against display_name for autocomplete). Returns display_name for UI, normalized_name for dedup	Yes
GET	/categories	List skill categories	Yes
POST	/	Add a new custom skill. **Restricted:** only allowed when called as part of an internship skill requirement or a confirmed resume extraction (i.e., the skill is being attached to an entity, not created in isolation). Backend normalizes the name, checks UNIQUE on normalized_name, validates: min 2 characters, max 100 characters, no special characters except `+`, `#`, `.`, `/` (for language names like C++, C#, Node.js, CI/CD). Rejects if normalized form matches an existing skill (returns the existing skill instead). Stores display_name as provided by the first creator.	Yes

Admin Routes (`/api/admin`)

Method	Endpoint	Description	Auth
GET	/dashboard-stats	Get all dashboard statistics	Admin
GET	/users	List all users (with filters, search, pagination)	Admin
GET	/users/:id	Get detailed user info	Admin
PUT	/users/:id	Edit a user	Admin
PUT	/users/:id/toggle-active	Activate or deactivate a user	Admin
DELETE	/users/:id	Permanently delete a user and all associated data. **Also cleans up uploaded files** (profile pictures, resume, company logos) from storage as best-effort. **Note:** deleting a user also deletes any conversations they participate in, which cascades to all messages in those conversations.	Admin
GET	/internships	List all internships (all statuses)	Admin
PUT	/internships/:id/approve	Approve an internship	Admin
PUT	/internships/:id/reject	Reject an internship with note	Admin
DELETE	/internships/:id	Permanently delete any internship. Same cascade behavior as employer delete	Admin
GET	/logs	Get system logs (with filters, pagination)	Admin
GET	/logs/export	Export logs as CSV (generated on demand, not stored)	Admin

Method	Endpoint	Description	Auth
POST	/reports/generate	Generate a report on demand. **Request body:** `{ reportType, startDate, endDate, filters }`. Queries the DB and returns the report data as JSON. No file is stored — the report is generated fresh each time. **Also inserts a row into the `report` table** as an audit log: records who generated it, the type, description, filters used, and timestamp. Response: `{ reportType, generatedAt, data: [...] }`.	Admin
GET	/reports/export	Export a report as CSV or PDF. **Query params:** same as generate plus `format=csv|pdf`. Generates on demand and returns the file as a download stream. No persistent file storage. **Also inserts a row into the `report` table** as an audit log (same as generate).	Admin
GET	/notifications	List admin notifications. **Paginated:** same params as student/employer notifications.	Admin
PUT	/notifications/:id/read	Mark notification as read	Admin
PUT	/notifications/read-all	Mark all notifications as read	Admin
PUT	/change-password	Change admin password (requires current password + new password). **Also increments `users.token_version`** to invalidate all existing JWT sessions.	Admin

---

EMAIL NOTIFICATIONS — TRIGGERS

Event	Recipient	Email Subject	When
Registration	New user	"Welcome to InternMatch!"	After successful registration
Internship Approved	Employer	"Your internship '{title}' has been approved"	Admin approves posting
Internship Rejected	Employer	"Your internship '{title}' needs revision"	Admin rejects posting
New Application	Employer	"New application for '{title}'"	Student applies
Application Status Changed	Student	"Update on your application to '{title}'"	Employer changes status
Password Reset	User	"Reset your InternMatch password"	User requests reset
Invitation to Apply	Student	"{company} invites you to apply for '{title}'"	Employer invites candidate
New Message	Recipient	"New message from {sender}"	New chat message (if not online)

Use Nodemailer with Gmail SMTP (free, up to 500/day) or Mailtrap for development/testing.

REAL-TIME FEATURES (Socket.IO)

Events:

Event Name	Direction	Payload	Purpose
`connection`	Auto	—	Socket connected + authenticated via JWT handshake. Server registers userId → socketId mapping from verified token.

Event Name	Direction	Payload	Purpose
`disconnect`	Auto	—	Clean up userId → socketId mapping
`message:send`	Client → Server	{ conversationId, content }	Send a chat message. Server validates sender is a participant in the conversation before saving/delivering.
`message:receive`	Server → Client	{ message object }	Deliver message to recipient in real time
`message:read`	Client → Server	{ conversationId }	Mark messages as read
`notification:new`	Server → Client	{ notification object }	Push new notification to user
`typing:start`	Client → Server	{ conversationId }	Show typing indicator
`typing:stop`	Client → Server	{ conversationId }	Hide typing indicator

Implementation:

- **Authentication:** Socket.IO connections MUST be authenticated via JWT. The client sends the JWT token in the handshake `auth` object: `io({ auth: { token: jwt } })`. The server verifies the token in the `io.use()` middleware before accepting the connection. Unauthenticated connections are rejected with `next(new Error('Authentication required'))`. The verified `userId` from the JWT is stored on the socket instance — the client does NOT send userId separately (that would be spoofable).
- Server uses **rooms per userId** (each user is auto-joined to a room named by their userId on connect) to support multiple tabs/devices
- When a message is sent, server saves to DB, then emits to recipient if online
- If recipient is offline, they get it on next page load via REST API
- Notifications work the same way: create in DB + push via socket if online

---

SECURITY SPECIFICATIONS

Concern	Implementation
Password Storage	bcryptjs with salt rounds = 12. Column should be understood as storing a hash, never a raw password.
Authentication	JWT tokens stored in `localStorage`. JWT payload includes `userId`, `role`, `tokenVersion`. No httpOnly cookies — this is the canonical auth model for the entire project including Socket.IO.
Token Invalidation	`users.token_version` is incremented on password change or admin-forced logout. Auth middleware checks `tokenVersion` in JWT against DB value — mismatch = reject token (401). This forces re-login after password changes.

Concern	Implementation
Active-User Check	Auth middleware MUST query `users.is_active` and `users.token_version` from the DB on every authenticated request. If `is_active = FALSE`, reject with 403 "Account deactivated." This is the only way to immediately block a suspended user whose JWT hasn't expired. The query is lightweight (PK lookup, single row) and acceptable for a university-scale project. Socket.IO `io.use()` middleware performs the same check on connection.
Authorization	Middleware checks role from JWT before allowing route access
Socket.IO Auth	JWT token sent in Socket.IO handshake `auth` object: `io({ auth: { token } })`. Server verifies token in `io.use()` middleware including `is_active` and `tokenVersion` checks. Unauthenticated or deactivated connections rejected. Socket mapping uses **rooms per userId** (not a single socketId map) to support multiple tabs/devices.
SQL Injection	Prepared statements with parameterized queries (mysql2 placeholders)
XSS	React auto-escapes output. Internship descriptions and company descriptions are **plain text only** — stored as TEXT, rendered with whitespace/line-break preservation (CSS `white-space: pre-wrap`), no HTML allowed, no sanitization needed.
CSRF	Not applicable — JWT is stored in localStorage (not cookies), so CSRF attacks cannot forge requests. CORS origin whitelist is still enforced.
File Upload	Validate file type by **both extension and magic bytes** (not just MIME header — MIME can be spoofed). PDF magic bytes: `%PDF`, DOCX: PK zip header `50 4B 03 04`, JPG: `FF D8 FF`, PNG: `89 50 4E 47`. Max size limits (5MB resume, 2MB profile picture/logo). Rename files with UUID. Never serve uploaded files using original filenames. Block path traversal characters in filenames.
Rate Limiting	express-rate-limit on auth routes (max 10 attempts per 15 min per IP). Additional rate limiting on Gemini API-triggering endpoints: resume upload (max 5 per hour per user), search queries (max 30 per minute per user), internship skill extraction (max 10 per hour per user)
Input Validation	express-validator on all POST/PUT routes; frontend inline validation
HTTPS	Required for production deployment (handled by hosting platform)
Open Redirect	The `/login?redirect=` parameter MUST be validated to only accept relative paths starting with `/`. Reject any absolute URLs, protocol-relative URLs (`//`), or URLs containing `://`. This prevents open redirect attacks.
Logging	All auth events, admin actions, and errors logged to system_log
Data Privacy	**Hard-delete policy (intentional design choice):** This is a university project, not a production system with legal retention requirements. When a user deletes their account, all owned data cascades (profile, internships, applications, messages, bookmarks, skills, embeddings). This means: if an employer deletes their account, students lose visibility of applications to that employer's internships. If a student deletes their account, employers lose that applicant from their lists. This is explicitly accepted as the intended behavior — no soft-delete, no archival. Anonymized audit/analytics records (system_log, internship_view) may remain with user_id = NULL after deletion. Admin cannot see raw passwords.
Logout	`POST /api/auth/logout` clears the JWT from the client (frontend removes token from localStorage). With stateless JWT there is no server-side session to destroy. To force-logout all sessions, increment `users.token_version` — the next request from any session with an old tokenVersion will be rejected by auth middleware.

RESPONSIVE DESIGN

All pages must work on mobile (< 640px), tablet (640px – 1024px), and desktop (> 1024px). Mobile uses single column layout with hamburger nav and stacked cards. Tablet uses two-column grid with collapsible sidebar. Desktop uses full sidebar + content area with multi-column grids. Exact breakpoints and implementation approach to be determined during frontend design phase.

DEPLOYMENT SPECIFICATIONS (Production-Ready)

Option A: Free Tier Deployment

Component	Service	Cost
Frontend	Vercel (free)	$0
Backend	Render.com (free tier) or Railway (free tier)	$0
Database	PlanetScale (free tier MySQL) or Railway MySQL	$0
File Storage	Cloudinary (free tier, 25GB) for resumes/logos	$0
Email	Gmail SMTP (free, 500/day) or Mailtrap (free, 500/month)	$0

Option B: Local Development

Component	How
Frontend	`npm start` → localhost:3000
Backend	`nodemon server.js` → localhost:5000
Database	MySQL Community Server locally
Files	Local `/uploads` directory
Email	Mailtrap (catches emails without sending real ones)

Environment Variables (`.env`)

```
# Server
PORT=5000
NODE_ENV=development
CLIENT_URL=http://localhost:3000

# Database
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=your_password
DB_NAME=internmatch

# JWT
JWT_SECRET=your_random_secret_key_here
JWT_EXPIRES_IN=24h
JWT_REMEMBER_EXPIRES_IN=30d

# Email
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=your_email@gmail.com
SMTP_PASS=your_app_password

# File Upload
UPLOAD_DIR=./uploads
MAX_FILE_SIZE=5242880

# Google Gemini AI
GEMINI_API_KEY=your_gemini_api_key_here
GEMINI_GENERATION_MODEL=gemini-2.5-flash-lite
GEMINI_EMBEDDING_MODEL=text-embedding-004

# Socket.IO
SOCKET_CORS_ORIGIN=http://localhost:3000

# Admin Seeding
ADMIN_DEFAULT_PASSWORD=Admin@123
```

Database Seeding (First-Time Setup)

On first deployment (or fresh database), the system requires seed data:

1. **Run Schema.sql** to create all tables. **Note:** `Schema.sql` is a `mysqldump` export (includes `FOREIGN_KEY_CHECKS=0`, lock statements, and dump boilerplate). It is safe for import/restore (`mysql < Schema.sql`) but is not a hand-written migration file. For version control, consider extracting a clean `CREATE TABLE` migration from it. The dump format handles the `student` ↔ `resume` cyclic FK dependency via `FOREIGN_KEY_CHECKS=0`, which a manual migration would need to replicate.
2. **Run seed script** (`backend/database/seed.sql` or a Node.js seed script) which must:
   - Insert the **default admin account**: email `admin@internmatch.com`, password bcrypt-hashed (the raw password is set via an environment variable `ADMIN_DEFAULT_PASSWORD` or defaults to `Admin@123` for local dev — must be changed in production). Role = `admin`. Inserts into both `users` and `admin` tables. **No `notification_preference` row for admins** — admin notifications are always delivered, admins have no preference toggles.

- Insert the **150+ seed skills** across all categories (programming, web, data, ai_ml, devops, mobile, design, soft_skill, other) with proper `display_name` and `normalized_name` values.

3. **Verification:** After seeding, the admin should be able to log in at `/login` with the default credentials and immediately change the password.

The seed script must be **idempotent** — running it multiple times should not create duplicates (use `INSERT IGNORE` or check-before-insert for skills, check if admin email already exists before inserting).

Aspect	Convention
Variables & Functions	camelCase (`matchScore` , `calculateMatch`)
React Components	PascalCase (`StudentDashboard` , `InternshipCard`)
Database columns	snake_case (`user_id` , `match_score`)
API routes	kebab-case (`/forgot-password` , `/read-all`)
File names (components)	PascalCase.jsx (`StudentDashboard.jsx`)
File names (backend)	camelCase.js (`authController.js`)
Constants	UPPER_SNAKE_CASE (`MAX_FILE_SIZE`)
Comments	JSDoc for all controller functions; inline comments for complex logic only
Error messages	Human-readable, no stack traces to client
Git commits	Conventional commits: `feat:` , `fix:` , `docs:` , `refactor:`
Git branches	`main` , `develop` , `feature/FR1-student-registration` , `bugfix/login-error`